COMP3225 - Natural Language Processing

Words

Natural Language Processing: study of interactions between computers and human language, in particular how to program computers to process and analyse large amounts of natural language data

Sentence	Unit of written language
Utterance	Unit of spoken language
Word Form	Inflected form as it appears in the corpus e.g. *said*
Lemma	Abstract root form of the word e.g. *say*
Function Word	Indicating a grammatical relationship e.g. *by*
Types	Number of distinct words in a corpus
Tokens	Total number of words

Corpus: collection of written texts having a specific author, language, time, place, and function
Token: symbol which determines how grammar and semantics can be understood
Morpheme: unit of language that cannot be divided further

<u>Regular Expressions</u>

- A regular expression is an algebraic notation for characterising a set of strings
- Operators show how symbols are joined together
- Capture groups find repeating groups in different locations
- Lookahead assertions match expressions without consuming them
- Anchors mark particular places in text
- Aliases a short versions of longer expressions

<u>Operators</u>

Operator	Meaning	Example
\	separator	/woodchuck/
[ ]	disjunction	/[abc]/

( )	precedence	/dogg(y|ies)/
{ }	repetition	/abc{3}/ or /abc{1,3}/
?	optional	/colou?r/
^	negation	/^a/
.	wildcard	/.txt/
|	disjunction	/cat|dog/
*	zero or many	/[0-9]*/
+	one or many	/[0-9]+/
*?	non-greedy zero or many	/[0-9]*?/
+?	non-greedy one or many	/[0-9]+?/
\(\) \1	capture group	/I have a \(\w+\), do you want a \1?/

Lookahead Assertions

Operator	Meaning	Example
?: ?= ?<= ?! ?<!	non capturing group look ahead look behind negative look ahead negative look behind	(?:word) (?=word) (?<=word) (?!word) (?<!word)

Anchors

Expression	Meaning
^	start of line
$	end of line
\b	word boundary
\B	non-word boundary

Aliases

Expression	Meaning	Expansion
\d	number	[0-9]
\w	alphanumeric or underscore	[a-z A-Z 0-9_]

\s	whitespace	[␣ \r\t\n\f]

Text Normalisation

Normalisation: words or tokens are put into standard format. For example, *uh-huh* becomes *uh huh*
Case Folding: all characters are mapped to lowercase
Lemmatisation: reducing words to their root. For example, *saying* becomes *say*

- The Porter stemmer removes affixes from words. It is based on a series of rewrite rules called a **cascade** which pipelines the output of each step into the next step

Sentence Segmentation: separation of text into sentences using punctuation, markers, and symbols

- ? and ! mark the end of sentences
- . can mark the end of a sentence, an abbreviation, or both
- . takes precedence, then ? and !, then other symbols
- UNIX can be used for naïve word tokenisation and normalisation
- The command tr -sc 'A-Za-z' '\n' < result.txt | sort | uniq finds and sorts all unique words in text

UNIX Commands

Command	Meaning	Example
tr	tokenise	tr -sc 'A-Za-z' '\n' adds a new line after each character sequence
sort	sort alphabetically	tr -sc 'A-Za-z' '\n' | sort
uniq	list unique elements	tr -sc 'A-Za-z' '\n' | sort | uniq

Patterns

Expression	Meaning
([A-Z]\.)+	abbreviations
\w+(-\w+)*	hyphenated words
\£?\d+(\.\d+)?	currency
\.\.\.	ellipsis

BPE (Byte Pair Encoding): form of data compression where the most common pair of consecutive bytes is replaced with a new byte

- starts with a vocabulary using characters [a-z] and tries to learn *k* new tokens
- repeats *k* times

- selects the two most frequently adjacent symbols e.g. (a,b)
- adds a new merged symbol AB to the dictionary
- replaces each A B with AB

<u>Minimum Edit Distance</u>

Minimum edit distance measures the similarity between strings, defined as the minimum number of editing operations, such as insertion, deletion, and substitution, needed to transform one string into another.

- the **alignment** shows the position of strings which provides the largest similarity
- i represents insertion
- d represents deletion
- s represents substitution

The **Levenshtein distance** between two sequences is the simplest weighting factor in which each operation has a cost of 1.

- Given two strings, **X** of length **i** and **Y** of length **j**, the edit distance is defined as **D(i,j)**

$$D(i,j) = \min \begin{cases} D(i-1,j)+1 \\ D(i,j-1)+1 \\ D(i-1,j-1)+1 \end{cases}$$

D(i,j) = min

```
function MIN-EDIT-DISTANCE(source, target) returns min-distance

  n ← LENGTH(source)
  m ← LENGTH(target)
  Create a distance matrix distance[n+1,m+1]

  # Initialization: the zeroth row and column is the distance from the empty string
      D[0,0] = 0
      for each row i from 1 to n do
          D[i,0] ← D[i-1,0] + del-cost(source[i])
      for each column j from 1 to m do
          D[0,j] ← D[0,j-1] + ins-cost(target[j])

  # Recurrence relation:
  for each row i from 1 to n do
      for each column j from 1 to m do
          D[i,j] ← MIN( D[i−1,j] + del-cost(source[i]),
                        D[i−1,j−1] + sub-cost(source[i], target[j]),
                        D[i,j−1] + ins-cost(target[j]))
  # Termination
  return D[n,m]
```

Evaluation

Evaluation Techniques

Extrinsic Evaluation: evaluates the performance of an NLP component by embedding it in an application and measuring its performance
Intrinsic Evaluation: measures the quality of an NLP component independently of its application

Steps in training and evaluation:

- Linguistic Theories
  - AI develops linguistic theories about patterns, grammars, and rules
- Statistical Patterns
  - AI infers statistical patterns and rules by examining text corpora
- Training
  - a training dataset is used to fit a model
  - typically 80% of data
  - uncovers dependencies and patterns, difference between noise and signals, significance of words and phrases
- Validation
  - a validation dataset is used to provide an unbiased evaluation of a model while tuning model hyperparameters
  - allows tuning of hyperparameters to optimise a model
  - typically 10% of data
- Testing
  - a test dataset is used to measure the validity and accuracy of a model
  - measures only the final performance of a model
  - typically 10% of data

The corpora size affects the quality of training on a model.

- a larger corpora means a more varied language and better training datda
- the relationship between the size of the vocabulary and the number of tokens N is given by the expression $|V| = kN\beta$ where k and β are positive and β is between 0 and 1
- β depends on the corpus size and context
- |V| is usually limited to ~50,000 words due to GPU memory restrictions
- an OOV (out-of-vocabulary) word is a word which doesn't appear in the training data
- ignoring OOV words can lead to overfitting as they are not analysed

K-Fold Cross-Validation: statistical procedure used to evaluate the skill of a model on limited data

- **Shuffle** the dataset randomly
- **Split** the dataset into K groups
- For each unique group:
  - take the group as the **test set**
  - take the remaining groups as the the **training set**

    - **fit** a model on the training set
    - **evaluate** the model on the test set
    - retain the **evaluation score** and discard the model
- Summarise the skill of the model as the **sample of model evaluation scores**

Random subsampling: evaluation technique similar to cross-validation, which selects subsets of data as the **test set** randomly rather than using K fixed-size groups. This improves randomisation, but may not be representative of all data

<u>Evaluation Metrics</u>

TP (True Positive): "yes" data is correctly predicted as "yes"
TN (True Negative): "no" data is correctly predicted as "no"
FP (False Positive): "no" data is incorrectly predicted as "yes"
FN (False Negative): "yes" data in incorrectly predicted as "no"

Accuracy =
proportion of **correct** predictions

$$\text{Accuracy} = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision =
proportion of **correct positive** predictions out of all **positive** predictions

$$\text{Precision} = \frac{TP}{TP+FP}$$

proportion of **correct positive** predictions out of all **positive** predictions

i.e. how many positive predictions were correct?

Recall =
proportion of **correct positive** predictions out of all **correct** predictions

$$\text{Recall} = \frac{TP}{TP+FN}$$

proportion of **correct positive** predictions out of all **correct** predictions

i.e. how many correct predictions were made?

F1 = harmonic mean of precision and recall

$$F1 = \frac{TP}{TP+\frac{FN+FP}{2}} \text{ or } \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$$

ROUGE: recall-based metric used to evaluate text summarisation

- a good text summary includes many repeated phrases
- a good text summary is similar to a human-generated version
- ROUGE-n is the proportion of n-grams / significant phrases which match reference summaries

BLEU: precision-based metric used to evaluate text translations

- measures how closely a machine translation matches a human-generated version
- measures how many n-word phrases match reference translations
- uses a function of n-gram precision over all sentences
- BP is the brevity penalty which penalises short translations, p is precision
- LF is the length of the reference translation, LS is the length of the system translation

BLEU = BP $\text{BLEU} = \text{BP} \times \sqrt[4]{\left(\prod_{n=1}^{4} p_n\right)}$

BP = min(1, exp(1 - LF/LS))

PERPLEXITY: statistical metric used to evaluate language models

- measures how well a vocabulary predicts target text
- uses the probability of all words in text appearing in the same order
- the perplexity of a set is its inverse probability normalized by the number of words
- the perplexity of a set W equals PP(W) where Π represents the product
- minimising the perplexity is equivalent to maximising the set probability

$$\text{PP(W)} = P(w_1 w_2 \ldots w_n)^{-\frac{1}{n}}$$

$$\text{PP(W)} = \sqrt[n]{\frac{1}{P(w_1 w_2 \ldots w_n)}}$$

$$\text{PP(W)} = \sqrt[n]{\prod_{i=1}^{n} \frac{1}{P(w_i | w_1 \ldots w_{i-1})}}$$

N-grams

Probabilities are useful for identifying words where the input is noisy, ambiguous, or broken up.

- Speech Recognition
  - recognition and translation of spoken language into written text
  - the model predicts which words are most likely to be spoken
- Spelling & Grammar Correction
  - checking for misspellings and grammar errors
  - the model predicts which spellings are more likely to be correct
- Machine Translation
  - translation of one language to another
  - the model predicts which translation is most likely to be correct
- Alternative Communication

- translation of physical movements such as eye gaze to spoken text
- the model predicts which words are most likely to be spoken

N-Gram: contiguous sequence of *n* items. The n-gram model calculates the probability of the next word in some text, given the previous n-1 words

- **Unigram:** single word e.g. "sequence"
- **Bigram:** two-word sequence e.g. "sequence of"
- **Trigram:** three-word sequence e.g. "sequence of words"

<u>N-gram Probabilities</u>

Assume P(w|h) is the probability of word *w* given the history *h*. Suppose the history is "*its water is so transparent that"*, *w* is *"the"*, and *C* means *count*:

$$P(A|B) = \frac{P(A)\,P(B|A)}{P(B)}$$

$$P(the\,|\,its\ water\ is\ so\ transparent\ that) = \frac{C(its\ water\ is\ so\ transparent\ that\ the)}{C(its\ water\ is\ so\ transparent\ that)}$$

Assume $P(w_{1:n})$ is the probability of a sequence $w_1, \ldots, w_n$. We can use **joint probability** to predict this value:

$P(w_1) = P(w_1)$

$P(w_1,w_2) = P(w_1|w_2) * P(w_1)$

$P(w_1,w_2,w_3) = P(w_3|w_1,w_2) * P(w_1,w_2)$

$P(w_1,w_2,w_3,w_4) = P(w_4|w_1,w_2,w_3) * P(w_1,w_2,w_3)$

=> $P(w_1, \ldots, w_n) = P(w_n|w_1, \ldots, w_{n-1}) * P(w_1, \ldots, w_{n-1})$

Applying the **chain rule of probability**:

$$\begin{aligned} P(w_{1:n}) &= P(w_1)P(w_2|w_1)P(w_3|w_{1:2})\ldots P(w_n|w_{1:n-1}) \\ &= \prod_{k=1}^{n} P(w_k|w_{1:k-1}) \end{aligned}$$

Assume $P(w_n|w_{n-1})$ is the probability of a bigram, i.e. the probability of a word w_n given all previous words by using only the probability of the previous word.

A Markov model is a probabilistic model which assumes it can predict the probability of some future event from its current state.

- A Markov assumption assumes that the probability of a word depends only on the previous word
- The bigram can be generalised to the n-gram, which looks *n-1* words into the past
- The N-gram approximation of the conditional probability of the next word in a sequence is given by:

$$P(w_n|w_{n-1}) \approx P(w_{n-N+1:n-1}|w_{n-1})$$

Given the bigram assumption for the probability of an individual word, we can calculate the probability of a sequence of words w1:n using ***substitution****:*

$$P(w_{1:n}) \approx \prod_{k=1}^{n} P(w_k|w_{k-1})$$

MLE (Maximum Likelihood Estimation): selection of the most suitable set of bigram parameters to maximise the performance of a model in predicting the nth word in some text

To calculate the bigram probability of a word *y* given the previous word *x*, we can calculate the count of the bigram *C(xy)* and normalise by the sum of all bigrams starting with *x*:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{\sum_w C(w_{n-1}w)}$$

We can simplify this equation by simplifying the denominator to the unigram count for w_{n-1}:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)}{C(w_{n-1})}$$

For the general case of MLE n-gram parameter estimation:

$$P(w_n|w_{n-N+1:n-1}) = \frac{C(w_{n-N+1:n-1}\ w_n)}{C(w_{n-N+1:n-1})}$$

Large n-gram probabilities must be stored as log probabilities to avoid numerical underflow. Adding in log space is equivalent to multiplying in linear space, so log probabilities are combined by addition. The original values can be restored by taking the exponential:

$$p_1 \times p_2 \times p_3 \times p_4 = \exp(\log p_1 + \log p_2 + \log p_3 + \log p_4)$$

<u>Limitations</u>

Unknown Words

- OOV (out-of-vocabulary) words appear in test sets
- open vocabulary systems replace OOV words with <UNK> tokens
- <UNK> tokens can be associated with OOV words based on their frequency
- <UNK> tokens can be used as a single word in the vocabulary
- <UNK> tokens often cause overfitting

Zero Probabilities

- models can predict a zero probability for n-grams which appear in test sets but not training and validation sets
- any joint probabilities with these n-grams will also be zero

Sparse Data

- Even when the vocabulary size |V| is large, only a tiny minority of possible n-grams exist in any corpus
- $P(w_k|w_{k-}1) = 0$ for most sequences w_{k-1}, w_k
- This produces sparse matrices
- More sparse as *n* increases

<u>Smoothing</u>

Smoothing prevents language models from assigning a probability of zero to unseen n-grams, by taking some probability from frequent n-grams and giving it to unseen n-grams.

In Laplace smoothing, one is added to all bigram counts before they are converted to probabilities, removing any zero values. The MLE for a unigram of word w_i is its count c_i normalised by the total number of tokens N.

$$P(w_i) = \frac{c_i}{N}$$

Laplace smoothing **adds one** to each word count and normalises by the vocabulary size V.

$$P_{Laplace}(w_i) = \frac{c_i + 1}{N + V}$$

It is more convenient to use the **adjusted count *c**** to allow easier comparison with other MLE values.

$$c_i^* = (c_i + 1)\frac{N}{N+V}$$

The level of smoothing is defined by the **relative discount d_c**, the ratio of adjusted counts to original counts.

$$d_c = \frac{c^*}{c}$$

For bigrams, the unigram count must be augmented by the vocabulary size *V*.

$$P^*_{Laplace}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+1}{C(w_{n-1})+V}$$

In add-k smoothing, a fractional amount *k* is added to all bigram counts before they are converted to probabilities.

$$P^*_{Add-K}(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n)+k}{C(w_{n-1})+kV}$$

Interpolation combines several n-gram estimators to give a weighted value. In linear interpolation, n-grams are combined by applying a weight *λ* where the sum of weights is 1.

$$\begin{aligned}\hat{P}(w_n|w_{n-2}w_{n-1}) &= \lambda_1 P(w_n|w_{n-2}w_{n-1}) \\ &\quad +\lambda_2 P(w_n|w_{n-1}) \\ &\quad +\lambda_3 P(w_n)\end{aligned}$$

Backoff estimates the probability of an n-gram which has no appearances by using a subset of the n-gram. For example, a trigram $w_{n-2}w_{n-1}w_n$ is replaced with the bigram $w_{n-1}w_n$ or the unigram w_n. If an n-gram has zero occurrences, the algorithm backs off to the (n-1)-gram until an occurrence is found.

Katz Backoff uses a parameter *α* to distribute the probability mass correctly.

$$P_{\mathrm{BO}}(w_n|w_{n-N+1:n-1}) = \begin{cases} P^*(w_n|w_{n-N+1:n-1}), & \text{if } C(w_{n-N+1:n}) > 0 \\ \alpha(w_{n-N+1:n-1})P_{\mathrm{BO}}(w_n|w_{n-N+2:n-1}), & \text{otherwise.} \end{cases}$$

BO = backoff, N = n-gram length

Kneser-Ney smoothing is a commonly used n-gram smoothing method. It subtracts an absolute discount d from each count, which helps to scale lower-order n-grams.

$$P_{\mathrm{AbsoluteDiscounting}}(w_i|w_{i-1}) = \frac{C(w_{i-1}w_i) - d}{\sum_v C(w_{i-1}\,v)} + \lambda(w_{i-1})P(w_i)$$

bigram *λ * unigram*

Kneser-Ney smoothing uses the continuation probability which estimates how frequently a word *w* appears as a unique continuation of an n-gram. It is calculated by the frequency of the n-gram containing *w* over the frequency of all n-grams with matching *n*. For example, *"Great Britain"* has a low continuation value because it only appears in a single context.

The size of the set of words v, that w appears after, *ie* where the count of (v,w) is greater than 0.

$$P_{\mathrm{CONTINUATION}}(w) = \frac{|\{v : C(vw) > 0\}|}{|\{(u', w') : C(u'w') > 0\}|}$$

Equation 3.32

The size of the set of adjacent word pairs (u', w') *ie* where the count of (u',w') is greater than 0.

Stupid Backoff is a simpler solution used for large-scale text corpora, which is inexpensive to train on large datasets and approaches the accuracy of Kneser-Ney smoothing as the size of the training set increases. Stupid Backoff does not discount higher-order probabilities. If a higher-order n-gram has no occurrences, the algorithm backs off to a lower-order n-gram weighted by a fixed weight *S*.

$$S(w_i|w_{i-k+1}^{i-1}) = \begin{cases} \frac{\mathrm{count}(w_{i-k+1}^{i})}{\mathrm{count}(w_{i-k+1}^{i-1})} & \text{if count}(w_{i-k+1}^{i}) > 0 \\ \lambda S(w_i|w_{i-k+2}^{i-1}) & \text{otherwise} \end{cases}$$

$$S(w) = \frac{count(w)}{N}$$

Parts of Speech

<u>POS Tagging</u>

A part of speech is a category to which a word is assigned in accordance with its syntactic functions. In English the main parts of speech are *noun, pronoun, adjective, determiner, verb, adverb, preposition, conjunction,* and *interjection.*

Parts of speech fall into two categories. Closed classes have a fixed membership where specific words belong. These are typically function words such as *of, it, and, you*. Open classes such as nouns and verbs are flexible and are continually growing to accommodate new words.

POS Tag: label assigned to tokens in a text corpus to indicate the part of speech and often other grammatical categories such as tense, number, and case
POS tagging: process of assigning a part of speech to each word in text. An **input sequence** $x_1, \ldots, x_n$ of **tokenised words** is converted into an **output sequence** $y_1, \ldots, y_n$ of **tags**.
Tagging is a **disambiguation** task because words are ambiguous and may have various possible tags. POS tagging resolves ambiguities and selects the correct POS tag for each word.

	Tag	Description	Example
Open Class	**ADJ**	Adjective: noun modifiers describing properties	*red, young, awesome*
	ADV	Adverb: verb modifiers of time, place, manner	*very, slowly, home, yesterday*
	NOUN	words for persons, places, things, etc.	*algorithm, cat, mango, beauty*
	VERB	words for actions and processes	*draw, provide, go*
	PROPN	Proper noun: name of a person, organization, place, etc..	*Regina, IBM, Colorado*
	INTJ	Interjection: exclamation, greeting, yes/no response, etc.	*oh, um, yes, hello*
Closed Class Words	**ADP**	Adposition (Preposition/Postposition): marks a noun's spacial, temporal, or other relation	*in, on, by under*
	AUX	Auxiliary: helping verb marking tense, aspect, mood, etc.,	*can, may, should, are*
	CCONJ	Coordinating Conjunction: joins two phrases/clauses	*and, or, but*
	DET	Determiner: marks noun phrase properties	*a, an, the, this*
	NUM	Numeral	*one, two, first, second*
	PART	Particle: a preposition-like form used together with a verb	*up, down, on, off, in, out, at, by*
	PRON	Pronoun: a shorthand for referring to an entity or event	*she, who, I, others*
	SCONJ	Subordinating Conjunction: joins a main clause with a subordinate clause such as a sentential complement	*that, which*
Other	**PUNCT**	Punctuation	, , ()
	SYM	Symbols like $ or emoji	$, %
	X	Other	asdf, qwfg

Tag	Description	Example	Tag	Description	Example	Tag	Description	Example
CC	coord. conj.	*and, but, or*	NNP	proper noun, sing.	*IBM*	TO	"to"	*to*
CD	cardinal number	*one, two*	NNPS	proper noun, plu.	*Carolinas*	UH	interjection	*ah, oops*
DT	determiner	*a, the*	NNS	noun, plural	*llamas*	VB	verb base	*eat*
EX	existential 'there'	*there*	PDT	predeterminer	*all, both*	VBD	verb past tense	*ate*
FW	foreign word	*mea culpa*	POS	possessive ending	*'s*	VBG	verb gerund	*eating*
IN	preposition/ subordin-conj	*of, in, by*	PRP	personal pronoun	*I, you, he*	VBN	verb past partici-ple	*eaten*
JJ	adjective	*yellow*	PRP$	possess. pronoun	*your, one's*	VBP	verb non-3sg-pr	*eat*
JJR	comparative adj	*bigger*	RB	adverb	*quickly*	VBZ	verb 3sg pres	*eats*
JJS	superlative adj	*wildest*	RBR	comparative adv	*faster*	WDT	wh-determ.	*which, that*
LS	list item marker	*1, 2, One*	RBS	superlatv. adv	*fastest*	WP	wh-pronoun	*what, who*
MD	modal	*can, should*	RP	particle	*up, off*	WP$	wh-possess.	*whose*
NN	sing or mass noun	*llama*	SYM	symbol	+,%, &	WRB	wh-adverb	*how, where*

<u>Hidden Markov Models</u>

A hidden Markov model (HMM) is a **probabilistic** sequence model. Given a sequence of **tokens**, it calculates the **probability distribution** over possible sequences of **tags** and selects the best sequence.

A Markov chain is a model showing the **probabilities** of **transitions** between random variables called **states**. A Markov chain makes an assumption that any future sequence only depends on the current state.

The hidden Markov model makes two assumptions. The **Markov assumption** assumes that the probability of a future state q_i depends only the previous state. The **output independence** assumes that an output observation o_i depends only on its state.

Markov Assumption: $P(q_i = a | q_1 \dots q_{i-1}) = P(q_i = a | q_{i-1})$

Output Independence: $P(o_i | q_1 \dots q_i \dots q_T, o_1 \dots o_i \dots o_T) = P(o_i | q_i)$

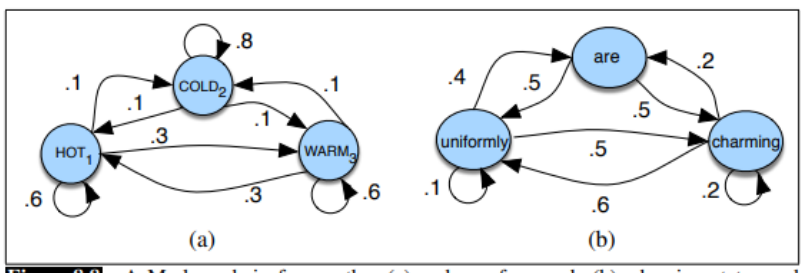


Figure 8.8 A Markov chain for weather (a) and one for words (b), showing states and transitions. A start distribution π is required; setting $\pi = [0.1, 0.7, 0.2]$ for (a) would mean a probability 0.7 of starting in state 2 (cold), probability 0.1 of starting in state 1 (hot), etc.

Formally, a Markov chain is defined by the following components:

$Q = q_1 \dots q_N$	set of states
$A = a_{11} \dots a_{N1} \dots a_{NN}$	transition probability matrix A each value *ij* represents the probability of moving from state *i* to state *j*
$\pi = \pi_1, \dots, \pi_N$	initial probability distribution over states is the probability the Markov chain will start at state *i* π_i is the probability the Markov chain will start at state *i*

A hidden Markov model contains **observed events** (input tokens) and **hidden events** (POS tags). A HMM is defined by the following components:

$Q = q_1 \dots q_N$	set of states

$A = a_{11} \dots a_{N1} \dots a_{NN}$	transition probability matrix each value *ij* represents the probability of moving from state *i* to state *j*
$O = o_1 \dots o_T$	set of observations from the vocabulary set of observations from the vocabulary $V = v_1 \dots v_V$
$B = b_i(o_t)$	emission probability matrix expresses the probability of an observation o_t being generated at state q_i
$\pi = \pi_1, \dots, \pi_N$	initial probability distribution over states is the probability the Markov chain will start at state *i* π_i is the probability the Markov chain will start at state *i*

<u>HMM Taggers</u>

A HMM uses two probabilities. *A* is the transition probability matrix representing the probability of a tag occurring given the previous tag. The model calculates the MLE of probabilities by counting tag occurrences over previous tag occurrences.

$$P(t_i|t_{i-1}) = \frac{C(t_{i-1}, t_i)}{C(t_{i-1})}$$

B is the emission probability matrix representing the probability of an observation being associated with a given tag. The model calculates the MLE of probabilities by counting word-tag pair occurrences over all tag occurrences.

$$P(w_i|t_i) = \frac{C(t_i, w_i)}{C(t_i)}$$

Decoding is the process of determining values of **hidden variables** (tags). Given an **input** *λ=(A,B)* and a set of **observations**, find the most probable set of **states**.

For POS tagging, the goal of decoding is to find the tag sequence $t_1 \dots t_n$ that is most likely given an observation sequence $w_1 \dots w_n$.

$$\hat{t}_{1:n} = \operatorname{argmax} P(t_1 \dots t_n | w_1 \dots w_n)$$

Using Bayes rule:

$$\hat{t}_{1:n} = \text{argmax} \frac{P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)}{P(w_1 \dots w_n)}$$

Simplifying by dropping the denominator:

$$\hat{t}_{1:n} = \text{argmax}\, P(w_1 \dots w_n | t_1 \dots t_n) P(t_1 \dots t_n)$$

A HMM tagger makes two assumptions. The first assumption states that the probability of a **word** occurring depends only on its **tag**.

$$P(w_1 \dots w_n | t_1 \dots t_n) \approx \prod_{i=1}^{n} P(w_i | t_i)$$

Second, the bigram assumption states that the probability of a **tag** occuring is dependent only on the **previous tag**.

$$P(t_1 \dots t_n) \approx \prod_{i=1}^{n} P(t_i | t_{i-1})$$

Using these assumptions, the following equation gives the most probable tag sequence for a bigram tagger. The equation states that the probability of a tag set $t_1 \dots t_n$ given a word set $w_1 \dots w_n$ is defined by the product of **emission** and **transition** values.

$$\hat{t}_{1:n} = \underset{t_1 \dots t_n}{\text{argmax}}\, P(t_1 \dots t_n | w_1 \dots w_n) \approx \underset{t_1 \dots t_n}{\text{argmax}} \prod_{i=1}^{n} \overbrace{P(w_i | t_i)}^{\text{emission}} \overbrace{P(t_i | t_{i-1})}^{\text{transition}}$$

<u>Viterbi Algorithm</u>

The Viterbi algorithm is an efficient algorithm for **HMM decoding** using **dynamic programming**.

- sets up a probability matrix with a **column** for **observations** and a **row** for each **state**
- each cell of the matrix represents the probability that the HMM is in state *j* after seeing the first *t* observations
- the value of each cell is calculated by taking the most probable path
- the cell probability $v_t(j) = max\ P(q_i \dots q_{t-1}, o_1 \dots o_t, q_t = j \mid \lambda)$
- the Viterbi probability is calculated by taking the most probable path extension to the current cell
- for state q_j at time *t*, the value of $v_t(j) = max\ (i)\ a_{ij}\, b_j\, (o_t)$

$v_{t-1}(i)$	previous Viterbi path probability
a_{ij}	transition probability from state q_i to state q_j
$b_j(o_t)$	observation probability for o_t given the state j

---

function VITERBI (*observations* of len *T*, *state-graph* of len *N*) **returns** *best-path, path-prob*

create a path probability matrix *viterbi [N,T]*

for *state s* from *1 to N* **do**

    viterbi [state,1] = *start probability* π_{state} × *emission probability* $b_s(o_1)$

for *time t* from *2 to T* **do**

    for *state s* from *1 to N* **do**

        viterbi [state, time] = max *viterbi [s', t-1]* × *transition probability* $a_{s'->s}$ × *emission probability* $b_s(o_t)$

best path probability = max *viterbi [s,T]*
best path pointer = argmax *viterbi [s,T]*
best path = path at state *bestpointer*, following pointers to states over time

return *best path, best path probability*

---

function VITERBI(*observations* of len T,*state-graph* of len N) **returns** *best-path*, *path-prob*

create a path probability matrix *viterbi[N,T]*
for each state s **from** 1 **to** N **do** ; initialization step
  $viterbi[s,1] \leftarrow \pi_s * b_s(o_1)$
  $backpointer[s,1] \leftarrow 0$
for each time step t **from** 2 **to** T **do** ; recursion step
  for each state s **from** 1 **to** N **do**
    $viterbi[s,t] \leftarrow \max_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
    $backpointer[s,t] \leftarrow \operatorname{argmax}_{s'=1}^{N} viterbi[s',t-1] * a_{s',s} * b_s(o_t)$
$bestpathprob \leftarrow \max_{s=1}^{N} viterbi[s,T]$; termination step
$bestpathpointer \leftarrow \operatorname{argmax}_{s=1}^{N} viterbi[s,T]$; termination step
bestpath ← the path starting at state *bestpathpointer*, that follows backpointer[] to states back in time
return *bestpath*, *bestpathprob*

Named Entity Recognition

<u>NER Tagging</u>

Named Entity: word or phrase referred to with a proper name such as a person, location, or organisation

Type	Tag	Sample Categories	Example sentences
People	PER	people, characters	**Turing** is a giant of computer science.
Organization	ORG	companies, sports teams	The **IPCC** warned about the cyclone.
Location	LOC	regions, mountains, seas	**Mt. Sanitas** is in **Sunshine Canyon**.
Geo-Political Entity	GPE	countries, states	**Palo Alto** is raising the fees for parking.

Named entity recognition (NER) aims to find spans of text that constitute proper names and tag them accordingly. BIO tagging is a method that allows us to treat NER as a sequence labelling task by capturing the boundaries of the sequence. I and O represent words inside and outside the entity, B and E represent the beginning and end, and S represents a single entity.

Variant	Tags	Meaning	Example
IO	I O	Inside Outside	Jane of United Airlines Holding discussed the Chicago route I-PER O I-ORG I-ORG I-ORG O O I-LOC O

BIO	B I O	Begin Inside Outside	Jane of United Airlines Holding discussed the Chicago route B-PER O B-ORG I-ORG I-ORG O O B-LOC O
BIOES	B I O E S	Begin Inside Outside End Single	Jane of United Airlines Holding discussed the Chicago route S-PER O B-ORG I-ORG E-ORG O O S-LOC O

<u>Conditional Random Fields</u>

Conditional Random Field (CRF): statistical modelling technique used to predict the most probable sequence of NER tags for a given sequence of words

$$\hat{Y} = \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} P(Y|X)$$

Linear Chain CRF: log-linear model that assigns a probability to an output tag sequence *Y*, out of all possible output sequences $\mathscr{Y}$, given an input word sequence *X*. Assuming a word sequence *X = x1,…,xn*, linear chain CRF produces a tag sequence *Y = y1,…,yn* by maximising the probability *P(Y|X)*:

$$p(Y|X) = \frac{\exp\left(\sum_{k=1}^{K} w_k F_k(X,Y)\right)}{\sum_{Y' \in \mathscr{Y}} \exp\left(\sum_{k=1}^{K} w_k F_k(X,Y')\right)}$$

or

$$p(Y|X) = \frac{1}{Z(X)} \exp\left(\sum_{k=1}^{K} w_k F_k(X,Y)\right)$$

$$Z(X) = \sum_{Y' \in \mathscr{Y}} \exp\left(\sum_{k=1}^{K} w_k F_k(X,Y')\right)$$

The ***K* functions *$F_K(X,Y)$*** are the global features for the sequences *X* and *Y*, calculated by taking the **sum of** local features at each position *i* in *Y*. Each local feature f_k uses the current token y_i, previous token y_{i-1}, input sequence *X*, and position *i*:

$$F_k(X,Y) = \sum_{i=1}^{n} f_k(y_{i-1}, y_i, X, i)$$

<u>Features</u>

Feature templates populate **features** with specific values. The template ⟨y_i,x_i⟩, ⟨y_i,y_{i-1}⟩, ⟨y_i,x_{i-1},x_{i+2}⟩ uses information from y_{i-1}, y_i, X, i. Word shape features represent the **abstract letter pattern** of words by mapping lowercase letters to *x,* uppercase letters to *X,* and numbers to *d*. For example, DC2021 -> XXdddd.

A gazetteer is a list of place names, often providing millions of entries for locations with detailed geographical and political information. Name lists containing organisations and products can also be used.

Typical features for a feature-based NER tagger

identity of w_i, identity of neighboring words
embeddings for w_i, embeddings for neighboring words
part of speech of w_i, part of speech of neighboring words
presence of w_i in a **gazetteer**
w_i contains a particular prefix (from all prefixes of length ≤ 4)
w_i contains a particular suffix (from all suffixes of length ≤ 4)
word shape of w_i, word shape of neighboring words
short word shape of w_i, short word shape of neighboring words
gazetteer features

Inference and Training

The best tag sequence $\hat{Y}$ for a given input X is given by:

- ignoring the *exp* function and denominator *Z(X)* as these are constant and do not affect comparisons
- expanding $F_K(X,Y)$ to $\sum_{i=1}^{n} f_k(y_{i=1}, y_i, X, i)$
- rearranging the expression

$$
\begin{aligned}
\hat{Y} &= \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} P(Y|X) \\
&= \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} \frac{1}{Z(X)} \exp\left(\sum_{k=1}^{K} w_k F_k(X,Y)\right) \\
&= \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} \exp\left(\sum_{k=1}^{K} w_k \sum_{i=1}^{n} f_k(y_{i-1}, y_i, X, i)\right) \\
&= \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} \sum_{k=1}^{K} w_k \sum_{i=1}^{n} f_k(y_{i-1}, y_i, X, i) \\
&= \underset{Y \in \mathscr{Y}}{\operatorname{argmax}} \sum_{i=1}^{n} \sum_{k=1}^{K} w_k f_k(y_{i-1}, y_i, X, i)
\end{aligned}
$$

To decode the optimal tag sequence we use the Viterbi algorithm, which works because a linear chain CRF depends only on the previous token at each time step. The algorithm fills in an ***N* × *T***

array with appropriate values while maintaining **backpointers**, then follows the pointers to retrieve the **optimal tag sequence**.

Lexical and Vector Semantics

<u>Lexical Semantics</u>

Lemma: root of a word e.g. *mouse*
Word Form: variant of a lemma e.g. *mice*
Word Sense: meaning of a lemma given its context, e.g. *mouse* as an animal or *mouse* as a device

Word Similarity: measure of similarity between the meanings of two words. Can be calculated by human judgement or by word co-occurrence
Word Relatedness: measure of association between words e.g. *cup* and *coffee*
Sentiment: positive or negative emotions suggested by words e.g. *happy, sad*

Semantic Field: set of words which cover a semantic domain and share structured relations. For example, words in the semantic field of *foods* include *bread, butter, cake, donuts*
Semantic Frame: set of words which denote perspectives or participants in an event. For example, the semantic frame for a *commercial transaction* includes *buy, sell, pay, money*

<u>Vector Semantics</u>

Vector semantics aim to represent a given word as a **point in a multidimensional space** using the **distributions of neighbouring words**. Vectors for representing words are called embeddings.

In a term-document matrix, each row represents a word and each column represents a document. The matrix has $|V|$ rows and D columns. Information retrieval (IR) is the task of finding a document d from D documents according to some query q represented by a vector $|V|$.

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	0	7	13
good	114	80	62	89
fool	36	58	1	4
wit	20	15	2	3

A term-term matrix is a $|V| \times |V|$ matrix encoding the occurences of a target word against several contexts. Each row represents a word and each column represents a possible context.

	aardvark	...	computer	data	result	pie	sugar	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

To measure the similarity between two words *v* and *w*, we can use cosine similarity to produce a value between 0 and 1. The dot product *v·w* is normalised by the magnitude of each vector.

$$\text{dot product}(\mathbf{v},\mathbf{w}) = \mathbf{v}\cdot\mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

$$\text{cosine}(\mathbf{v},\mathbf{w}) = \frac{\mathbf{v}\cdot\mathbf{w}}{|\mathbf{v}||\mathbf{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

TF-IDF (Term Frequency-Inverse Document Frequency) evaluates the relevance of a term to a document.

- Term Frequency (TF) - frequency of a term t in a document d, typically represented logarithmically
- Document Frequency (DF) - number of documents a term occurs in
- Inverse Document Frequency (IDF) - number of documents divided by document frequency
- TF-IDF - calculated by multiplying TF and IDF.

$$\text{tf}_{t,d} = \log_{10}(\text{count}(t,d)+1)$$

$$\text{idf}_t = \log_{10}\left(\frac{N}{\text{df}_t}\right)$$

$$w_{t,d} = \mathrm{tf}_{t,d} \times \mathrm{idf}_t$$

Pointwise Mutual Information (PMI) measures the association between two words by counting their co-occurrences. Given two words *w* and *c*, we measure how often they occur together compared to how often they occur independently. Positive Pointwise Mutual Information (PPMI) replaces negative values with 0.

$$\mathrm{PMI}(w,c) = \log_2 \frac{P(w,c)}{P(w)P(c)}$$

$$\mathrm{PPMI}(w,c) = \max(\log_2 \frac{P(w,c)}{P(w)P(c)}, 0)$$

Word2Vec

<u>Embeddings and Skip-Grams</u>

A word embedding is a learned representation for text where words that have the same meaning have a similar representation

- **dense** - number of dimensions *d* ranging from 50-1000
- **static** - single embedding per word that does not change
- **binary** - uses logistic regression to predict whether a word A is associated with word B

Skip-grams are a generalisation of n-grams in which terms do not need to be consecutive, but can be separated by a number of terms. Formally, a *k*-skip-*n*-gram is a subsequence of **length *n*** where terms occur at most ***k* terms away** from each other.

- target words and context words are taken as positive data points
- random sampling generates negative data points
- logistic regression is used to train the classifier
- the model learns weights for embeddings

<u>Word2Vec</u>

Word2Vec: neural network model that learns word associations from a given text corpus. After training, it can detect synonymous words using vector embeddings

Given an input of a target word *w* and a window size *L* to find context words *c*, the classifier will return the probability that *c* is a context word for *w*:

…lemon a [tablespoon of apricot jam, a] pinch…
c1 c2 c3 w c4

P(+|w,c) = probability *c* is a context word for *w*
P(-|w,c) = probability *c* is not a context word for *w*

The similarity between words is calculated using embeddings normalised by the logistic function to return a value between 0 and 1:

Similarity(w,c) ≈ *c* · *w*

$$\sigma(x) = \frac{1}{1 + exp(-x)}$$

$$P(+|w,c) = \sigma(c \cdot w) = \frac{1}{1 + exp(-c \cdot w)}$$

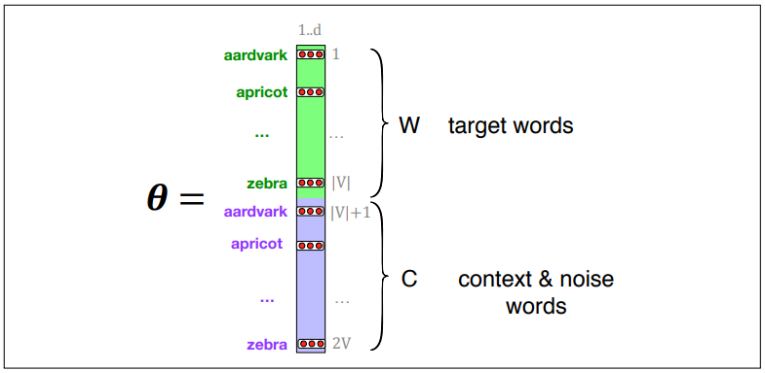
$$P(-|w.c) = 1 - P(+|w,c) = \sigma(-c \cdot w) = \frac{1}{1 + \exp(c \cdot w)}$$

Assuming that all context words are independent, the probability for all context words *c* in the window *L* can be calculated by taking the product:

$$P(+|w, c_{1:L}) = \prod_{i=1}^{L} \sigma(c_i \cdot w)$$

$$\log P(+|w.c_{1:L}) = \sum_{i=1}^{L} \log \sigma(c_i \cdot w)$$

In summary, Word2Vec trains a probabilistic classifier that takes a target word *w* and its context window of *L* words $c_{1:L}$ and assigns a probability based on the similarity between the context window and target word. This is based on the logistic function and the dot product of word embeddings. Word2Vec keeps embeddings for **every target word *w*** and **every context word *c***.



<u>Learning Skip-Gram Embeddings</u>

Skip-gram embeddings are learned by starting with random vectors and iteratively shifting the values for each word *w* to become more like the embeddings of associated words. Positive examples are **context words associated with the target word**. Negative examples are noise words found using **random sampling**.

…lemon a [tablespoon of apricot jam, a] pinch…

positive examples +

w	c_{pos}
apricot	tablespoon
apricot	of
apricot	jam
apricot	a

negative examples -

w	c_{neg}	w	c_{neg}
apricot	aardvark	apricot	seven
apricot	my	apricot	forever
apricot	where	apricot	dear
apricot	coaxial	apricot	if

Random words are chosen according to their weighted unigram frequency $P_\alpha(w)$ where α is a weight. This is set to $\alpha=0.75$ by default as $P_\alpha(w) > P(w)$ for rare words:

$$P_\alpha(w) = \frac{count(w)^\alpha}{\sum_{w'} count(w')^\alpha}$$

Given the set of positive and negative training instances, the goal of the algorithm is to adjust these embeddings to:

- maximise the similarity of the **target word** and **context word** pairs *(w,c+)* from positive samples
- minimise the similarity of **noise word** and **context word** pairs *(w,c-)* from negative samples

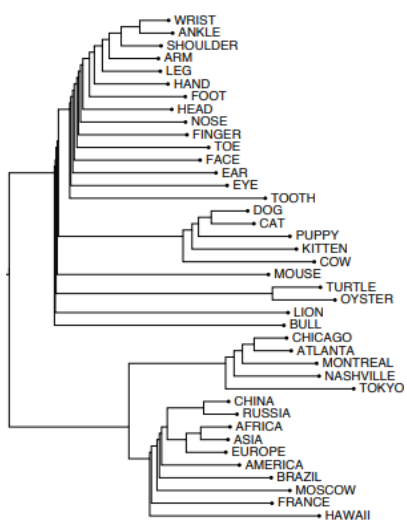
Given a positive word-context pair *(w,c+)* and *k* noise words $c\text{-}_1 \ldots c\text{-}_k$, these goals can be expressed as the loss function *L* to be minimised. The first term expresses that the **positive context word *c+*** should have a **high probability of being a neighbour**, while the second term

expresses that the **negative noise words *c-*** should have a **high probability of being a non-neighbou**r.

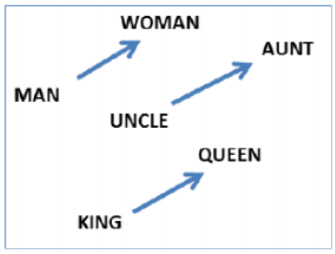
$$
\begin{aligned}
L_{CE} &= -\log\left[P(+|w,c_{pos})\prod_{i=1}^{k}P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log P(-|w,c_{neg_i})\right] \\
&= -\left[\log P(+|w,c_{pos}) + \sum_{i=1}^{k}\log\left(1-P(+|w,c_{neg_i})\right)\right] \\
&= -\left[\log \sigma(c_{pos}\cdot w) + \sum_{i=1}^{k}\log \sigma(-c_{neg_i}\cdot w)\right]
\end{aligned}
$$

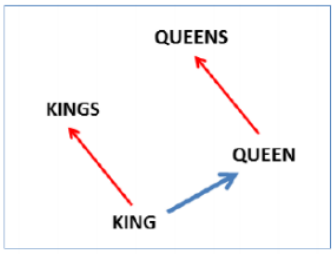
<u>Visualising Embeddings</u>

- Hierarchical Clustering
  - shows a hierarchical representation of the similarity of words in an embedding space
  - projects *n* dimensions down to 2 dimensions



- Analogy
  - shows the relational similarity of words
  - displayed using the parallelogram model
  - uses the analogy *a is to b as c is to ?*





Bias in Embeddings

Embeddings encode the bias within training sets used to compute them. A historical corpus is likely to encode the bias of its time.

- Bias Amplification
    - bias where embeddings exaggerate patterns and make them more extreme
    - implicit bias includes racism, ageism, sexism etc.
- Allocation Harm
    - bias resulting in unfair real world outcomes
- Representational Harm
    - harm caused by a system demeaning some social or racial groups

Debiasing is the process of manipulating embeddings to remove harmful bias.

Recurrent Neural Networks

Neural Units

A neural unit takes a **set of** real numbers and a bias term as input, computes the weighted sum of inputs, and outputs the result. Given a set of inputs $x_1 \ldots x_n$, a unit has a set of weights $w_1 \ldots w_n$ and a bias term b, which produces a weighted sum z:

$$z = \sum_i w_i x_i + b$$

Using vector notation:

$$z = w \cdot x + b$$

Instead of using the linear function z, neural units can apply a non-linear function f to z. The output of this function is the activation value a:

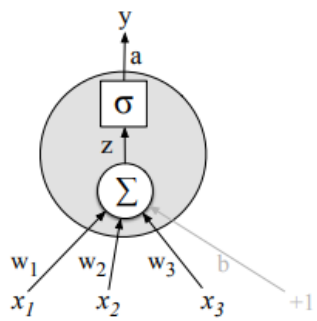
$$y = a = f(z)$$

Using the sigmoid function, the output of a neural unit is:

$$sigmoid = \sigma(x) = \frac{1}{1 + e^{-x}}$$

$$y = \sigma(w \cdot x + b) = \frac{1}{1 + \exp(-(w \cdot x + b))}$$

Schematic of a basic neural unit:



Neural Networks

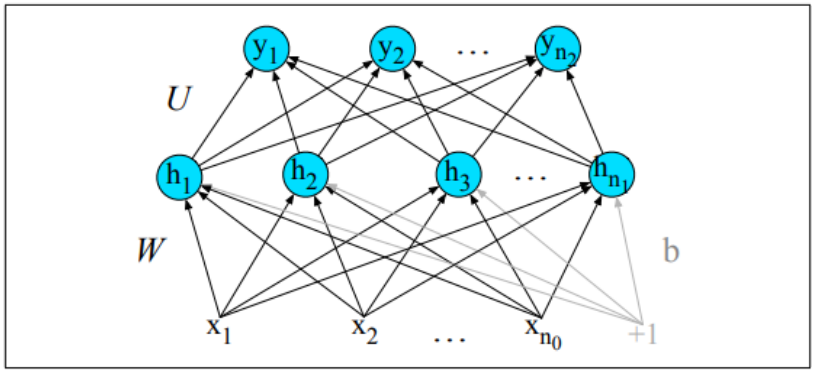
An artificial neural network (ANN) is a computing system based on biological neural networks.

- consists of many **neurons** connected together by **edges**
- neurons use **mathematical functions** and **weights** to compute their outputs
- **thresholds**, **aggregate signals**, and **layers** can be added to artificial neural networks

A multi-layer perceptron (MLP) consists of an input layer, hidden layers, and an output layer. Each layer is fully connected to the next layer. The input layer is composed of input neurons, while the hidden and output layers are composed of TLUs. Bias neurons are also included in the network. When an ANN contains many hidden layers it is called a deep neural network (DNN).

Feed-Forward Neural Networks

A feed-forward network is a multilayer neural network where connections are acyclic (no cycles). The outputs from units in each layer are passed to units in the next layer. No outputs are passed back to previous layers. In a fully connected feedforward network, information only travels forwards and each layer is fully connected to the next.



A fully connected feed-forward network takes inputs $x_1 \ldots x_n$ and a bias term b. It has a hidden layer with units $h_1 \ldots h_n$ and an output layer with units $y_1 \ldots y_n$. The weights and bias term are combined into a weight matrix W. The sigmoid function is used in the hidden layer to output the output h:

$$h = \sigma(Wx + b)$$

The hidden layer has a weight matrix U, which is multiplied by the input to produce an output z:

$$z = Uh$$

The softmax function normalises the output to a binary vector following a probability distribution where all values sum to 1:

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{d} e^{z_j}}$$

The final output y of the network is the normalised vector z:

$$y = softmax(z)$$

To calculate the output of the Nth layer in a neural network:

- the output $z^{[i]}$ is calculated using its weight matrix $W^{[i]}$, bias term $b^{[i]}$, and the output $a^{[i-1]}$ of the previous layer

$$z^{[i]} = W^{[i]} a^{[i-1]} + b^{[i]}$$

- the previous output $a^{[i]}$ is calculated using an activation function $g^{[i]}$ and the output $z^{[i]}$ of that layer

$$a^{[i]} = g^{[i]} z^{[i]}$$

where square bracket notation [i] represents a layer number

Each layer of the network is represented in the following form:

$$\begin{aligned} z^{[1]} &= W^{[1]} a^{[0]} + b^{[1]} \\ a^{[1]} &= g^{[1]}(z^{[1]}) \\ z^{[2]} &= W^{[2]} a^{[1]} + b^{[2]} \\ a^{[2]} &= g^{[2]}(z^{[2]}) \\ \hat{y} &= a^{[2]} \end{aligned}$$

The algorithm for computing the forward step in an n-layer feed-forward network given the input vector $a^{[0]}$ is given by:

$$\begin{aligned} &\textbf{for } i \textbf{ in } 1..\mathrm{n} \\ &\quad z^{[i]} = W^{[i]} a^{[i-1]} + b^{[i]} \\ &\quad a^{[i]} = g^{[i]}(z^{[i]}) \\ &\hat{y} = a^{[n]} \end{aligned}$$

<u>Training Neural Networks</u>

A feed-forward neural network is a supervised learning task using an observation *x* and target *y* at each step. This system produces an estimate of the true output value $\hat{y}$. The training procedure learns the parameters $W^{[i]}$ and $b^{[i]}$ for each layer *i* to make $\hat{y}$ closer to *y*.

The loss function measures the prediction error by calculating the difference between true values and prediction values. The cross-entropy loss function for logistic regression is:

$$L_{CE}(y, \hat{y}) = -\log p(y|x) = -[y \log \hat{y} + (1-y) \log(1-\hat{y})]$$

For a neural network with a probability distribution vector *C*, the cross-entropy loss is:

$$L_{CE}(y, \hat{y}) = -\sum_{i=1}^{C} y_i \log \hat{y}_i$$

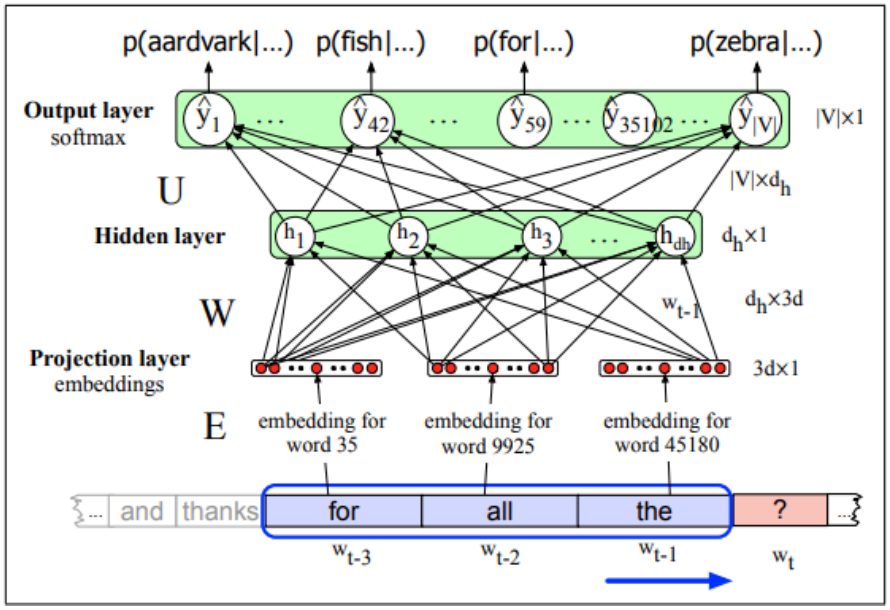
Neural Language Models

Neural language models use a moving window to move over a given text. Assuming the vector embeddings have already been pretrained, each group of *N* embeddings are concatenated as a one-hot vector. (A one-hot vector is a vector where one element is equal to 1 and all other elements are set to zero). The embedding matrix *E* is a dictionary of embeddings that contains a *d*-dimensional column vector for each word. At each time step *t*, *N* embeddings are taken and multiplied by *E*, then all three are converted into a *d*-dimensional embedding. Each of these is multiplied by the weight matrix *W* and put into an activation function to produce the hidden layer *h*. This is multiplied by its weight matrix *U* and put into the softmax function to produce the final output.

This network predicts the probability that the next word w_t will be a given vocabulary word *Vi* and uses the window size *N=3*.

- Select 3 embeddings from *E*
  - takes the next three words and creates three one-hot vectors
  - multiplies each vector by the embedding matrix *E*
  - each row of *E* is a word embedding and the input is a one-hot column vector x_i
  - so the projection layer for input *w* is $Ex_i = e_i$
  - the three embeddings are concatenated
- Multiply by *W*
  - multiply the *d*-dimensional embedding by the weight matrix *W*
  - pass through the activation function to get the hidden layer *h*
- Multiply by U
  - multiply the hidden layer *h* by *U* to get the output layer
- Apply softmax
  - use the softmax function to produce the final output layer

    use the softmax function to produce the final output layer $\hat{y}$
  - each node *I* in the output layer estimates the probability $P(w_t = i|w_{t-1},w_{t-2},w_{t-3})$



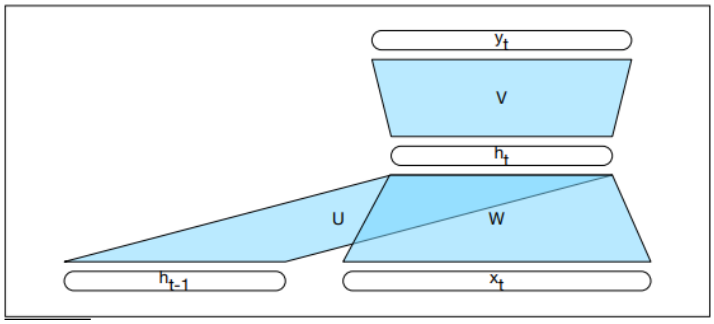
To train a neural language model, the parameters *W, U, E, b* must be set using gradient descent. Backpropagation is performed on the network to compute the gradient and set *W, U* and *E*. Training proceeds by starting with random weights and iteratively adjusting them as the network scans the input text. The cross-entropy loss for each word w_t is:

$$L_{CE} = -\log p(w_t | w_{t-1} \dots w_{t-n+1})$$

<u>Recurrent Neural Networks</u>

A recurrent neural network is a neural network that contains a cycle within its connections. Recurrent units are dependent on their own output as an input.

An Elman network or simple recurrent network is a single unit containing an input layer x_t, hidden layer h_t, and output layer y_t. The hidden layer contains a recurrent connection where the previous output becomes the current input. *W* is the weight matrix for the input layer. *U* is the matrix of hidden weights, which connects the hidden layer from the previous time step to the current time step. *V* is the matrix of output weights.



The activation function from the hidden layer h_t is used to compute an output y_t for input x_t. The hidden layer is calculated by passing the weighted values of the previous time step and the input of the current time step into the activation function. The output layer is calculated by passing the weighted values from the hidden layer into some activation function:

$$h_t = g(Uh_{t-1} + Wx_t)$$

$$y_t = f(Vh_t)$$

For soft classification (using class probabilities) the softmax function can be used:

$$y_t = softmax(Vh_t)$$

The algorithm for computing the forward step in an n-layer recurrent feed-forward network is given by:

function FORWARDRNN(x, *network*) **returns** output sequence y

$h_0 \leftarrow 0$
for $i \leftarrow 1$ **to** LENGTH(x) **do**
  $h_i \leftarrow g(U\ h_{i-1} + W\ x_i)$
  $y_i \leftarrow f(V\ h_i)$
return y

Modern deep learning models unroll the recurrent neural network in time to compute loss. The **learning template** specifies the network structure, parameters, weight matrices, activation and output functions. Given an input sequence, the model generates an unrolled feed-forward network over time. Regular **inference** or **backpropagation** can now be performed on the network.

<u>RNN Language Models</u>

RNN language models process inputs one word at a time. They predict the next word in the sequence using the current word and hidden state. The input *x* consists of word embeddings represented as one-hot vectors and output predictions *y*. At each step, the model uses the embedding matrix *E* to retrieve the embedding for the current word and combines it with the previous hidden layer to compute the new hidden layer. This passes through the output layer and softmax function:

$$e_t = E^T x_t$$

$$h_t = g(Uh_{t-1} + We_t)$$

$$y_t = softmax(Vh_t)$$

Given the output *y*, the probability of a word *i* occurring next is its corresponding *y* component:

$$P(w_{1:n}) = \prod_{i=1}^{n} P(w_i | w_{1:i-1}) = \prod_{i=1}^{n} y_{w_i}^{i}$$

In teacher forcing, an RNN language model is trained using the cross-entropy loss function:

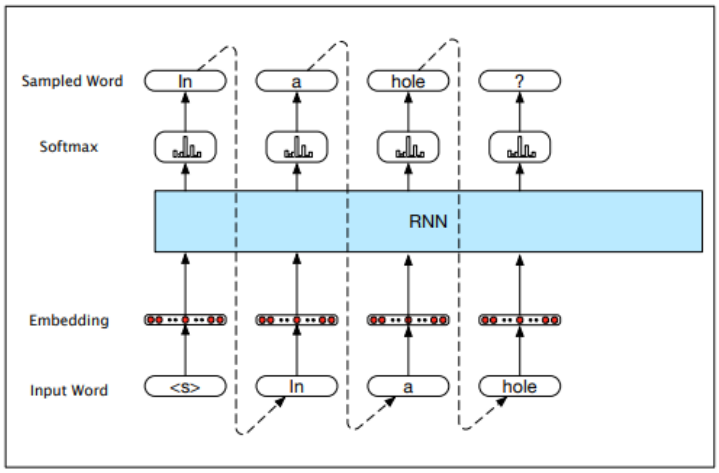
$$L_{CE} = -\sum_{w \in V} y_w^t \, log \, \hat{y}_w^t$$

For language modelling, the cross-entropy loss is determined by the probability of the next word:

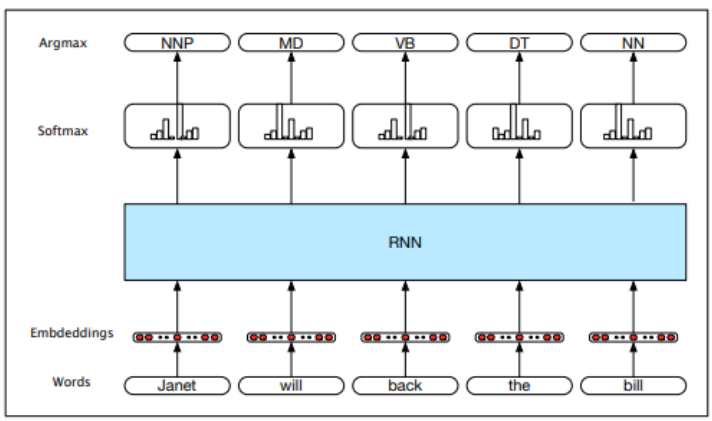
$$L_{CE}(\hat{y}^t, y^t) = -\log \hat{y}_{w_{t+1}}^t$$

<u>Applications of RNN Language Models</u>

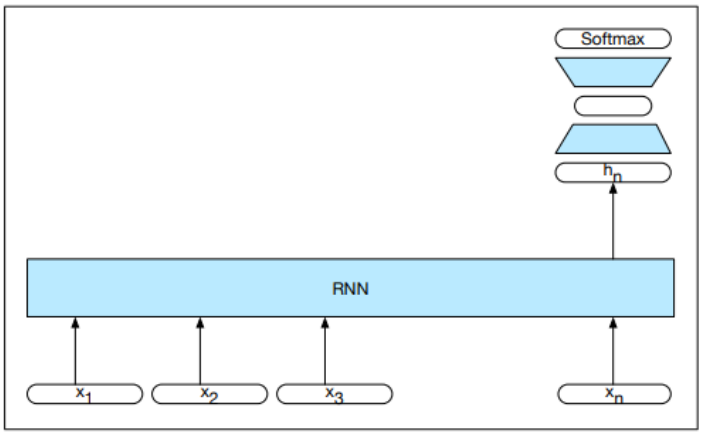
- Autoregressive Generation (Text Generation)
    - takes an input sequence *x* and outputs the next word *y*
    - uses pre-trained word embeddings and cross-entropy
    - looks at word associations to find related words



- Sequence Labelling (POS Tagging)
  - takes an input sequence *x* and outputs POS tag probabilities *y*
  - uses pre-trained word embeddings and cross-entropy
  - the RNN block represents an unrolled recurrent neural network consisting of an input layer, hidden layer, and output layer at each time step, with weight matrices *U, V, W*

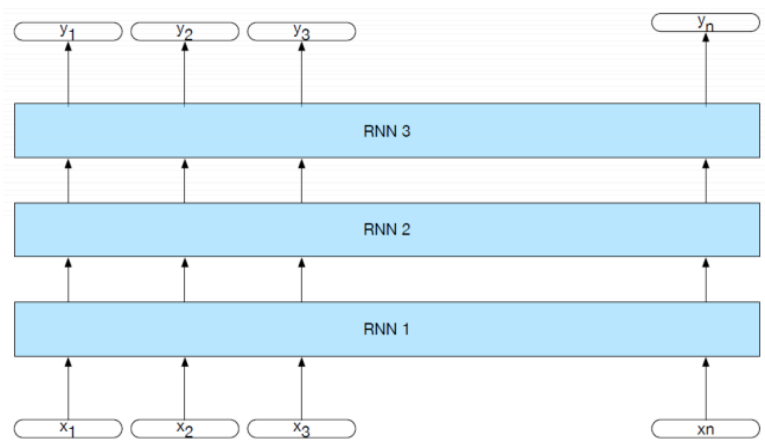


- Sequence Classification
  - takes an input sequence *x* and outputs a class probability *y*
  - input is processed one word at a time, generating a new hidden layer at each time step
  - the final hidden layer h_n is passed into the softmax function to find class probabilities



<u>Stacked and Bidirectional RNNs</u>

In a stacked RNN, the output of one RNN is used as the input to another. Adding layers boosts the performance of the stack and allows abstraction of inputs.



In a simple recurrent network, the hidden state at time *t* represents **all the information in the network** at that time step, as a result of some function *f* applied to the inputs:

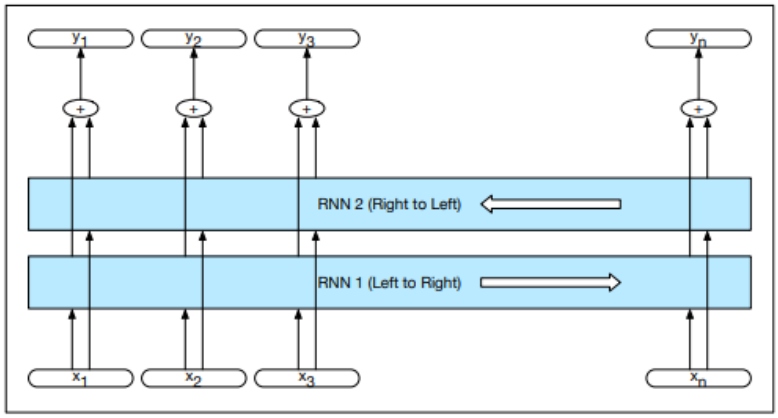
$$h_t^f = RNN_{forward}(x_1^t)$$

If the network is trained in reverse, the hidden state at time *t* represents **the information to the righ**t of the current input:

$$h_t^b = RNN_{backard}(x_1^n)$$

A bidirectional RNN combines the **forward and backward networks** into a single representation that captures the **left and right context** of an input at each point in time:

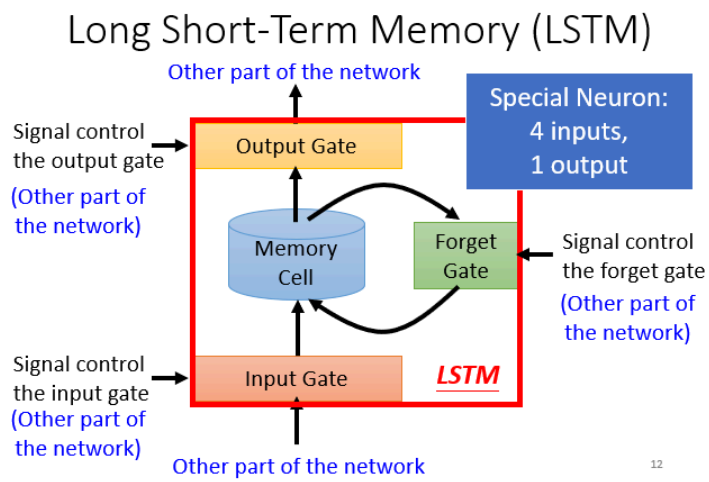
- $h_t = h_t^f \oplus h_t^b$

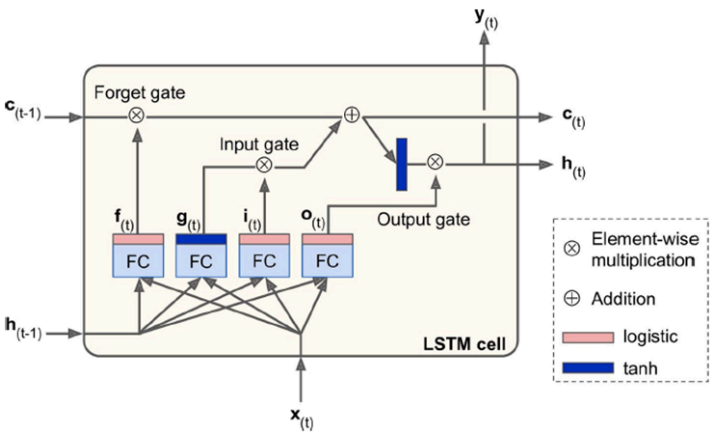


Sequence Processing

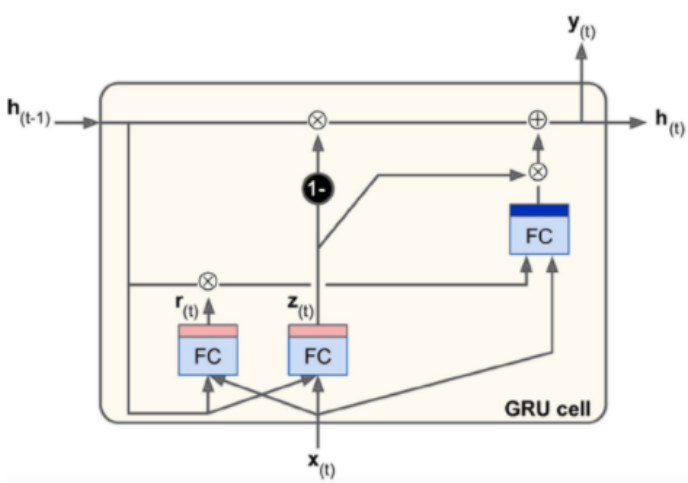
<u>Memory Gates</u>

- LSTM (Long Short-Term Memory)
  - RNN architecture containing feedback connections, composed of a memory cell, input gate, output gate, and forget gate
  - Remembers values over time while three gates regulate the flow of information into/out of the cell
  - Can add or remove information to the memory cell
  - **Input Gate:** provides input data from previous layers
  - **Output Gate:** provides output data to future layers
  - **Memory Cell:** internal memory of the neuron
  - **Forget Gate:** controls signals





- GRU (Gated Recurrent Unit)
  - Simplification of LSTM, but has fewer parameters and lacks an output gate
  - Contains a memory cell, input gate, and forget gate



Attention

Self-attention allows a network to extract and use information from a given context without using recurrent connections. To compute an attention score, it compares an item to a collection of other items to calculate its relevance in the current context. This uses a dot product comparison:

$$score(x_i, x_j) = x_i \cdot x_j$$

Once all the scores are calculated, they are normalised with the softmax function to create a vector of weights indicating the proportional relevance of each input *i*:

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{d} e^{z_j}}$$

$$\alpha_{ij} = softmax\left(score(x_i, x_j)\right) = \frac{\exp(score(x_i, x_j))}{\sum_{k=1}^{i} \exp(score(x_i, x_j))}$$

Given the score vector α, it generates an output y_i by calculating the sum of weighted inputs (multiplying the inputs and relevance values):

$$y_i = \sum_{j \leq i} \alpha_{ij} x_j$$

Transformers

Query: **current focus of attention** that is being compared to all other previous inputs
Key: **preceding input** that is being compared to the query
Value: **embedding** used to compute the output for the current focus of attention

Transformers use three sets of weights W^Q, W^K, and W^V which are multiplied with inputs x_i:

$$q_i = W^Q x_i; \;\; k_i = W^K x_i; \;\; v_i = W^V x_i$$

$$Q = W^Q X; \;\; K = W^K X; \;\; V = W^V X$$

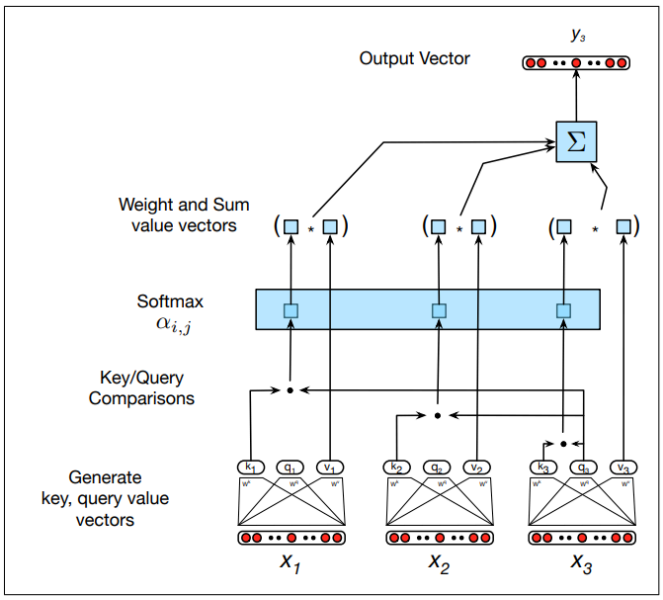
The self-attention score between the current input x_i and some element x_j is the dot product of the query and key vectors, normalised by the number of dimensions:

$$score(x_i, x_j) = \frac{q_i \cdot k_j}{\sqrt{d_k}}$$

$$SelfAttention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The final softmax calculation remains the same but is based on the weighted sum of the value vectors *v*:

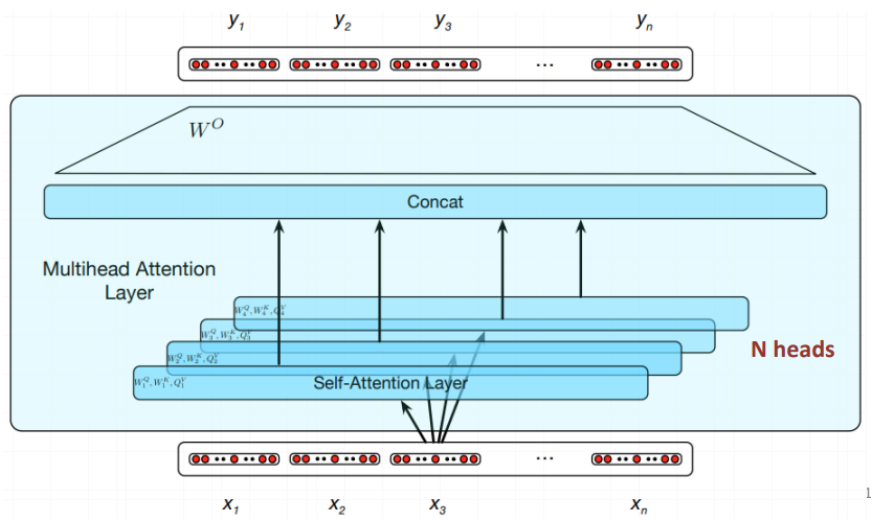
$$y_i = \sum_{j \leq i} \alpha_{ij} v_j$$



<u>Self-Attention Networks</u>

A transformer is an encoder-decoder architecture that handles long-range sequences by using attention to compute the **relevance of each input** to produce an output. Transformers map sequences of input vectors $x_1 \ldots x_n$ to output sequences $y_1 \ldots y_n$ of the same length. They are made up of stacks of network layers consisting of **simple layers, self-attention layers, feed-forward networks, and connections**.

Multi-head self-attention layers are **groups of self-attention layers** called heads which work in parallel at the same depth in a model, each with their own set of parameters. Given these distinct parameters, each head can **learn different aspects of the text at the same level of abstraction**.



Positional Embeddings

Transformer inputs are combined with positional embeddings so that the network can keep track of each element and its location.

- transformers do not encode the positions of words, so use positional embeddings
- these can be learned during training or generated using a positional encoding function
- positional embeddings have the same dimensions as word embeddings
- the positional embedding and word embedding are summed to give the full embedding

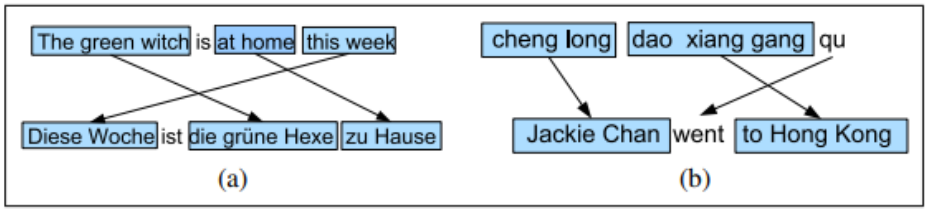
Machine Translation

Language Divergence and Typology

Some aspects of language are universal and hold true for every language. These include greetings, pronouns, politeness, questions, nouns and verbs. However, languages also differ in many ways. The study of linguistic similarities and differences is called linguistic typology.

Word order typology is the difference in the order of verbs, subjects, and objects in different languages. Languages such as English, French, and German are SVO (Subject-Verb-Object), Hindi and Japanese are SOV (Subject-Object-Verb), and Irish and Arabic are VSO (Verb-Subject-Object).

Subject-Verb-Object (SVO)	English, German, French, Mandarin ...
Subject-Object-Verb (SOV)	Japanese, Hindi ...
Verb-Subject-Object (VSO)	Arabic, Irish ...

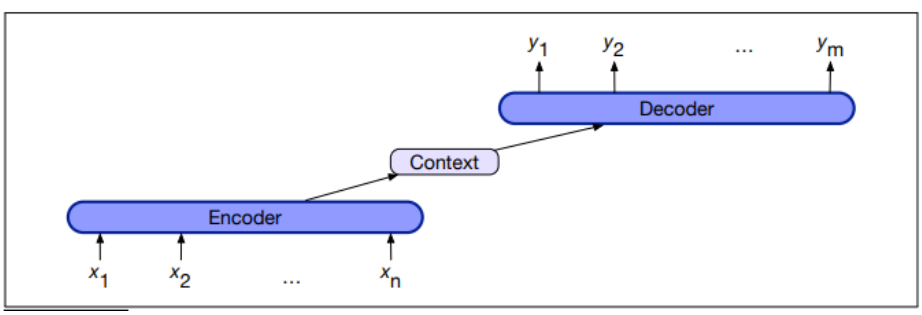


Referential density refers to the frequencies of omission in different languages. **Cold** languages can omit pronouns and require the reader to infer the subject of the sentence, while **hot** languages are explicit.

<u>The Encoder-Decoder Model</u>

Encoder-decoder networks are sequence-to-sequence networks capable of generating translated output sequences. The encoder takes the input sequence and creates a contextualised representation of it using word embeddings. The context is passed to the decoder, which generates a translated output sequence.

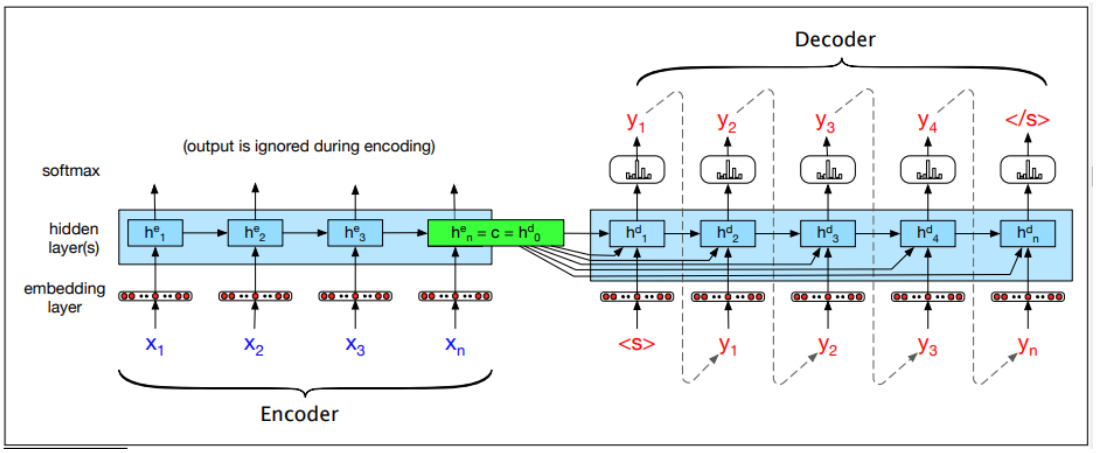
- Encoder
  - accepts an input sequence x_1^n
  - generates a corresponding sequence of contextualised representations h_1^n
  - can be implemented using LSTM, GRU, transformers, stacks
- Context
  - the context vector *c* is a function of h_1^n
  - *c* is passed to the decoder as input
- Decoder
  - accepts *c* as input and generates a sequence of hidden statesh_1^m
  - usesh_1^m to generate a sequence of output statesy_1^m



Using the source text *x* and target *y*, the encoder-decoder model **predicts the next target word in sequence *y* based on the previous target word and source sequence *x***:

$$p(y|x) = p(y_1|x)p(y_2|y_1,x)p(y_3|y_1,y_2,x)...P(y_m|y_1,...,y_{m-1},x)$$

This encoder-decoder network translates from English to Spanish. To translate the source text, we run it through the encoder to generate hidden states and the context vector. The decoder uses the context and the previous hidden state to generate each successive word in the translation.



The encoder takes an input sequence *x* and generates contextualised representations for each time step. The final state h^e_n is the context *c*, which is passed to the decoder. The decoder uses *c* as the first hidden state h^d_0. It generates a sequence of outputs h^d_t until an end-of-sequence marker is generated. Each hidden state is trained on the previous output, previous hidden state, and context *c*.

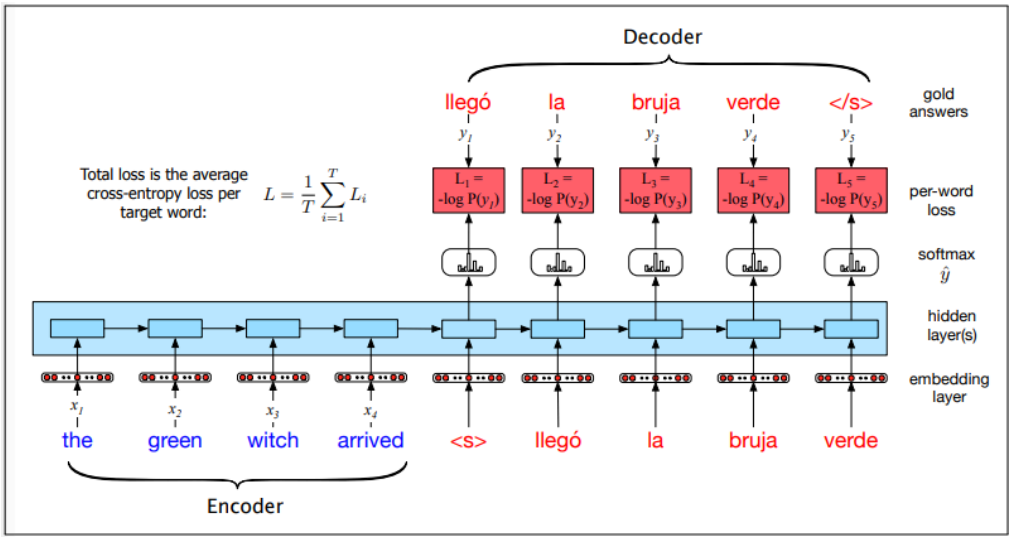
$$\begin{aligned} c &= h^e_n \\ h^d_0 &= c \\ h^d_t &= g(\hat{y}_{t-1}, h^d_{t-1}, c) \\ z_t &= f(h^d_t) \\ y_t &= \text{softmax}(z_t) \end{aligned}$$

The output *y* at each time step is passed through a softmax function over the set of all possible outputs to calculate the most likely output using the argmax:

$$\hat{y}_t = \text{argmax}_{w \in V} P(w|x, y_1 ... y_{t-1})$$

<u>Training the Encoder-Decoder Model</u>

Encoder-decoder networks are trained using source-target pairs as training data. In training, it is common to use teacher forcing in the decoder. This means that the system is forced to use the correct target word as the next input x_{t+1}, rather than relying on its predicted output $\hat{y}_t$.

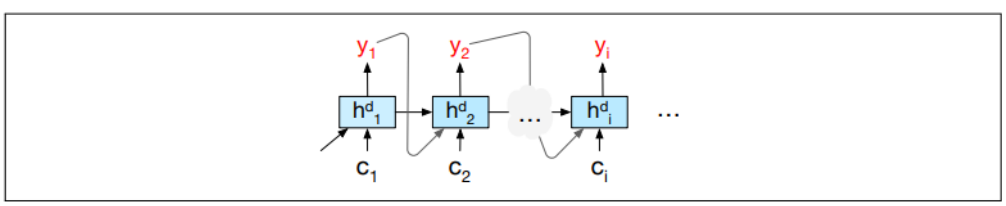


<u>Attention</u>

The context *c* may act as a bottleneck for the encoder, as it must equally represent all data in the source text. The attention mechanism is a more accurate method of implementing long-term memory using all states in the encoder.

The context *c* is a fixed-length vector which is a function of the hidden states i.e. $c = f(h_1^n)$. The function calculates the weighted sum of all the encoder's hidden states h_1^n. The context is dynamically derived at each point during decoding. This vector c_i is generated again at each step *i* and takes all of the hidden states into account.

$$h_i^d = g(\hat{y}_{i-1}, h_{i-1}^d, c_i)$$



Attention also looks at the relevance of each encoder state to the decoder state h^d_{i-1}. The relevance is captured by computing the dot-product attention score:

$$score(h^d_{i-1}, h^e_j) \;=\; h^d_{i-1} \cdot h^e_j$$

= decoder hidden state, = encoder hidden state

h^d_{i-1}*= decoder hidden state,* h^e_j*= encoder hidden state*

The score vector across all encoder hidden states provides the relevance of each decoder state to the current step of the decoder. These are normalised with the softmax function to create a weight vector α_{ij} that calculates the proportional relevance of each encoder hidden state h^e_j to the previous decoder hidden state h^d_{i-1}.

$$\alpha_{ij} \;=\; \text{softmax}(score(h^d_{i-1}, h^e_j) \;\; \forall j \in e)$$

$$= \frac{\exp(score(h^d_{i-1}, h^e_j)}{\sum_k \exp(score(h^d_{i-1}, h^e_k))}$$

Given the distribution in α, the fixed-length context vector c_i can be computed by taking the weighted average over all encoder hidden states:

$$c_i \;=\; \sum_j \alpha_{ij}\, h^e_j$$

<u>Beam Search</u>

Beam search is a heuristic search algorithm that explores a graph by expanding the most promising node in a limited set. Beam search uses breadth-first-search to build its search tree. At each level, it generates the successors of each state and sorts them in terms of heuristic cost. It selects the β best states to expand at each level, where β is the beam width. Rather than performing greedy search, which only selects the most promising node, beam search selects β possible nodes at each step. In the first step of decoding, it computes a softmax distribution over the vocabulary, assigning a probability to each word. It selects the β best options and expands each node, where the selected nodes are called the hypotheses.

Each of $k \times V$ hypotheses is scored by $P(y_i|x,y_{<i})$, which is the probability of the current word multiplied by the probability of the path leading to it. The beam prunes hypotheses down to the β best hypotheses until an end token </s> is generated, indicating a complete output. The solution is removed from the tree and beam search continues until the beam width is reduced to 0.

We can use the chain rule of probability to break down *P(y|x)* into the product of each word probability given its context, which we can convert into a sum of logs:

$$\begin{aligned} score(y) &= \log P(y|x) \\ &= \log\left(P(y_1|x)P(y_2|y_1,x)P(y_3|y_1,y_2,x)...P(y_t|y_1,...,y_{t-1},x)\right) \\ &= \sum_{i=1}^{t} \log P(y_i|y_1,...,y_{i-1},x) \end{aligned} \tag{11.20}$$

This can be normalised to a fixed length by dividing by the number of words:

$$score(y) = -\log P(y|x) \;=\; \frac{1}{T}\sum_{i=1}^{t} -\log P(y_i|y_1,...,y_{i-1},x)$$

Constituency Grammars

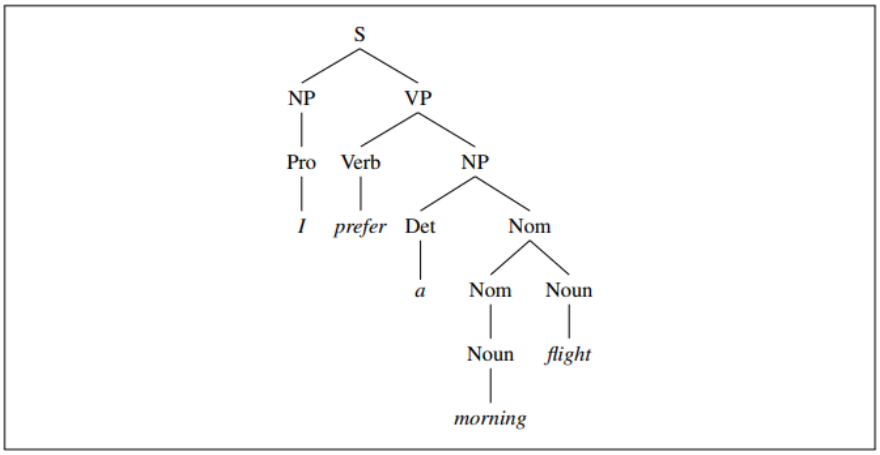
<u>Constituency</u>

Syntactic constituency is the idea that groups of words can behave as single units called constituents. Words are grouped into phrases such as noun phrases or verb phrases and are generated using grammars.

A context-free grammar consists of a set of **symbols** and a set of **rules or productions** of the form *A* → *a* where *A* is non-terminal and *a* is a sequence of symbols. Productions are written in BNF:

NP → *Det Nominal*
NP → *ProperNoun*
Nominal → *Noun* | *Nominal Noun*

Derivations of context-free grammars can be represented as a parse tree. The root node dominates nodes below it. A parse tree often begins with the start symbol *S*.



<u>Grammar Rules</u>

- A sentence can consist of a noun phrase and verb phrase

  S → NP VP *I prefer a morning flight*

- A verb phrase consists of a verb following by other constituents

  VP → Verb NP *prefer a morning flight*
  VP → Verb NP PP *leave Boston in the morning*
  VP → Verb PP *leave on Thursday*

- A preposition is followed by a noun phrase

  PP → Preposition NP *from England*

- A nominal is a head noun with extra modifiers

  Nominal → Noun *flight*
  Nominal → Nominal Noun *the flight*
  Nominal → Num Nominal *6 flights*

- A determiner is a word that appears before a noun such as *a, any, some, the, this, those, these*. It can also be a possessive expression such as *my, your, their*, or *Reece's*.

Grammar Rules			Examples
S	→	*NP VP*	I + want a morning flight
NP	→	*Pronoun*	I
	|	*Proper-Noun*	Los Angeles
	|	*Det Nominal*	a + flight
Nominal	→	*Nominal Noun*	morning + flight
	|	*Noun*	flights
VP	→	*Verb*	do
	|	*Verb NP*	want + a flight
	|	*Verb NP PP*	leave + Boston + in the morning
	|	*Verb PP*	leaving + on Thursday
PP	→	*Preposition NP*	from + Los Angeles

- A declarative sentence contains a subject noun phrase followed by a verb phrase

  S → NP VP — *The flight leaves at 6pm*

- An imperative sentence contains a verb phrase with no subject

  S → VP — *Show me the flight at 6pm*

- A yes-no question contains an auxiliary verb, subject noun phrase, and verb phrase

  S → Aux NP VP — *Are any flights available today?*

- A wh-subject-question contains a wh-word *(who,whose,when,where,what,which how,why),* noun phrase and verb phrase

  S → Wh-NP VP — *What flights leave at 6pm?*

- A wh-non-subject question contains a wh-phrase and two subjects

  S → Wh-NP Aux NP VP — *What flights do you have at 6pm?*

A wh-non-subject-question contains a long-distance dependency because the *Wh-NP* constituent is far away from the predicate *VP.*

A clause represents a *'complete thought'* and is made up of two or more of the following components:

- Subject - what the clause is about
- Verb - action done to the object by the subject
- Object - person, place, thing or idea
- Complement - extra subject or object which completes the phrase

- Adverbial - adjunct (additional information)

<u>Treebanks</u>

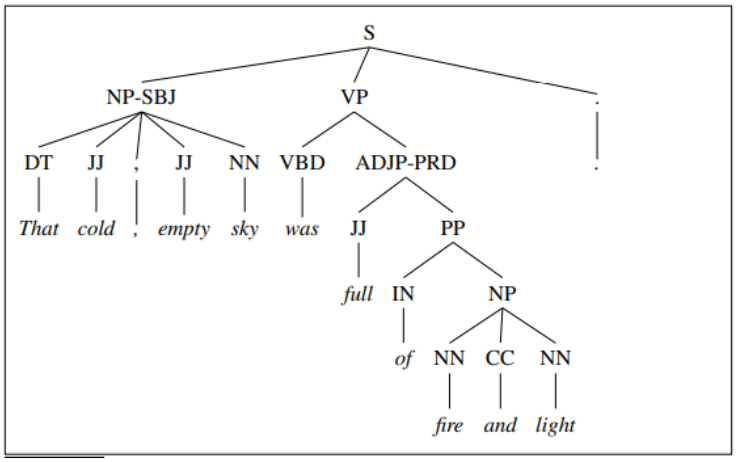
A treebank is a syntactically annotated corpus. In a treebank, every sentence in the collection is paired with a corresponding parse tree.

```
((S
   (NP-SBJ (DT That)
     (JJ cold) (, ,)
     (JJ empty) (NN sky) )
   (VP (VBD was)
     (ADJP-PRD (JJ full)
       (PP (IN of)
         (NP (NN fire)
           (CC and)
           (NN light) ))))
   (. .) ))
```

(a)

```
((S
   (NP-SBJ The/DT flight/NN )
   (VP should/MD
     (VP arrive/VB
       (PP-TMP at/IN
         (NP eleven/CD a.m/RB ))
       (NP-TMP tomorrow/NN )))))
```

(b)



A *-NONE-* token represents a long-distance dependency or a syntactic movement. For example a speech phrase such as *"Let's go," he said* separates the speech from the non-spoken phrase.

Treebanks tend to implicitly encode a grammar has rules are repeated over the data.

Grammar	Lexicon
$S \rightarrow$ *NP VP .*	*PRP* $\rightarrow$ *we* | *he*
$S \rightarrow$ *NP VP*	*DT* $\rightarrow$ *the* | *that* | *those*
$S \rightarrow$ *" S " , NP VP .*	*JJ* $\rightarrow$ *cold* | *empty* | *full*
$S \rightarrow$ *-NONE-*	*NN* $\rightarrow$ *sky* | *fire* | *light* | *flight* | *tomorrow*
NP $\rightarrow$ *DT NN*	*NNS* $\rightarrow$ *assets*
NP $\rightarrow$ *DT NNS*	*CC* $\rightarrow$ *and*
NP $\rightarrow$ *NN CC NN*	*IN* $\rightarrow$ *of* | *at* | *until* | *on*
NP $\rightarrow$ *CD RB*	*CD* $\rightarrow$ *eleven*
NP $\rightarrow$ *DT JJ , JJ NN*	*RB* $\rightarrow$ *a.m.*
NP $\rightarrow$ *PRP*	*VB* $\rightarrow$ *arrive* | *have* | *wait*
NP $\rightarrow$ *-NONE-*	*VBD* $\rightarrow$ *was* | *said*
VP $\rightarrow$ *MD VP*	*VBP* $\rightarrow$ *have*
VP $\rightarrow$ *VBD ADJP*	*VBN* $\rightarrow$ *collected*
VP $\rightarrow$ *VBD S*	*MD* $\rightarrow$ *should* | *would*
VP $\rightarrow$ *VBN PP*	*TO* $\rightarrow$ *to*
VP $\rightarrow$ *VB S*	
VP $\rightarrow$ *VB SBAR*	
VP $\rightarrow$ *VBP VP*	
VP $\rightarrow$ *VBN PP*	
VP $\rightarrow$ *TO VP*	
SBAR $\rightarrow$ *IN S*	
ADJP $\rightarrow$ *JJ PP*	
PP $\rightarrow$ *IN NP*	

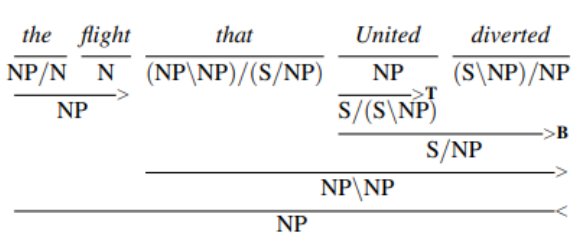
A lexical head is the most grammatically important word in a phrase. For example, *Noun* is the head of *NP* and *Verb* is the head of *VP*. Each context-free rule is associated with a lexical head which appears in the parse tree.

Combinatory Categorical Grammar is a generative grammar that allows both left-to-right and word-by-word composition. It consists of a set of **symbols**, **categories**, and a set of **rules or productions**.

The notation ***(X / Y)*** denotes a function that takes *Y* as an argument to the **right** and returns *X*. The notation ***(X \ Y)*** is the same but takes its argument from the **left**. The following rules apply:

$$(X / Y) \;\; Y \rightarrow X$$
$$Y \; (X \backslash Y) \rightarrow X$$

A combination of more complex rules allows derivations to be formed:



Constituency Parsing

Syntactic Parsing: technique by which tokenised POS-tagged text is assigned a structure that reveals relationships between tokens governed by syntax rules

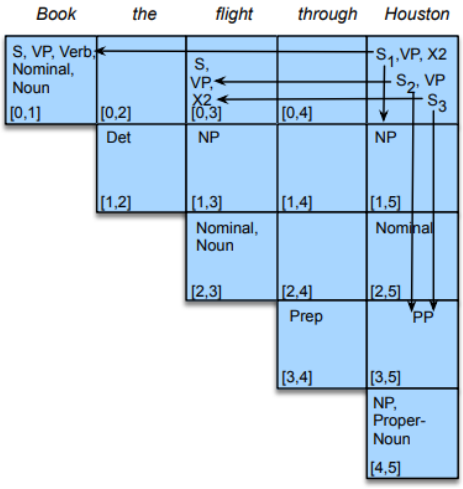
Structural ambiguity occurs when a grammar can assign more than one structure to a sentence.

- Attachment ambiguity - a constituent can be attached in multiple places
- Coordination ambiguity - phrases can be conjoined by conjunctions in multiple ways

<u>CKY Parsing</u>

The CKY algorithm is a dynamic programming technique operating on context-free grammars given in CNF to derive the most probable parse trees. It fills in a table of sub-problems to complete the original problem as a whole. CKY indexes sentences using **fenceposts** between tokens e.g. $_0$ *Book* $_1$ *that* $_2$ *flight* $_3$. The parse table is filled in left-right and bottom-up. Each **superdiagonal** row contains the POS tags for the current word. The other cells are filled by analysing phrases within the sentence.

The **outermost loop** iterates over the **columns** and the **next loop** iterates over the **rows**. The **innermost loop** ranges over cells where a substring may be split. At each **cell** *[i,j]* the algorithm considers whether the contents of the two cells can be combined via the grammar rules. If a rule exists, the left non-terminal is entered into the table.



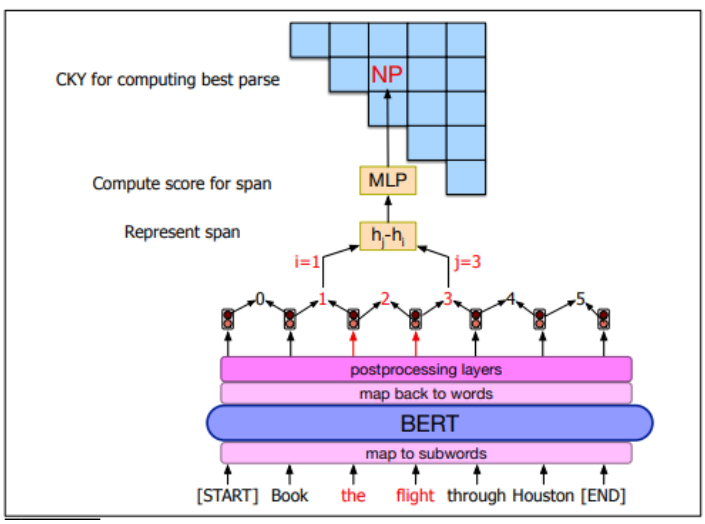
```
function CKY-PARSE(words, grammar) returns table

  for j ← from 1 to LENGTH(words) do
     for all {A | A → words[j] ∈ grammar}
           table[j − 1, j] ← table[j − 1, j] ∪ A
     for i ← from j − 2 down to 0 do
        for k ← i + 1 to j − 1 do
           for all {A | A → BC ∈ grammar and B ∈ table[i,k] and C ∈ table[k, j]}
                 table[i,j] ← table[i,j] ∪ A
```

<u>Span-Based Neural Consistency Parsing</u>

Span-Based Neural Consistency Parsing or neural CKY **disambiguates** among the possible parse trees to select the most probable tree. It trains a neural classifier to assign scores to constituents, then uses CKY to find the best-scoring parse tree.

Neural CKY architecture:

- input tokens are embedded using a pretrained model such as BERT
- embeddings are passed through post-processing layers such as transformers
- encoder outputs y_t are used to compute a span score
- span vector v is passed through an MLP classifier containing two fully connected layers and a ReLU
- the dimensionality of the output is equal to the number of possible non-terminal symbols
- the MLP outputs a score for each symbol



A parse tree T is a set of $|T|$ spans where the tth span starts at position i_t and ends at position j_t with label l_t. Given a span $[i,j]$ with non-terminal symbol k, the classifier assigns a score $s(i,j,l)$ to the span.

$$T = \{(i_t, j_t, l_t) : t = 1, \dots, |T|\}$$

The parser uses these scores to compute the score for the whole parse tree *s(T)* by summing over the scores of its constituent spans.

$$s(T) = \sum_{(i,j,l) \in T} s(i, j, l)$$

The **most probable parse** tree is the tree with the maximum score. The algorithm greedily selects the highest scoring label for each span. This has a 95% accuracy in finding valid trees.

$$\hat{T} = argmax\ s(T)$$

The variant CKY algorithm finds the full parse tree. It defines $S_{best}(i,j)$ as the score of the best subtree spanning *(i,j)*.

For spans of length 1:

$$S_{best}(i, i+1) = \max s(i, i+1, l)$$

For other spans *(i,j)*:

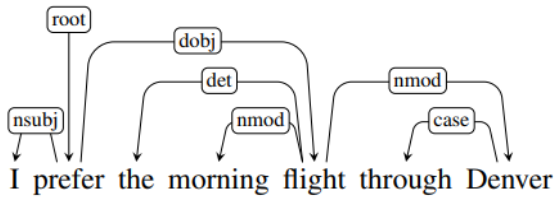
$$S_{best}(i, j) = \max s(i, j, l) + \max[s_{best}(i, k) + s_{best}(k, j)]$$

Dependency Parsing

<u>Dependency Grammars & Relations</u>

Dependency grammars are defined by a set of directed binary grammatical relations that hold among words in a sentence. They are useful for languages that are morphologically rich and have free word order. They can also be used in conference resolution, question answering, and information extraction.

Relations are represented by labelled directed arcs from heads to dependents. This is a **typed dependency structure** as labels are drawn from a fixed set of relations. The root node marks the head of the tree.



Dependency relations define labels for head and dependent word groups. **Clausal** relations involve a predicate (often a verb) and modifiers attached to it. **Nominal** relations involve modifiers for nominal (often noun) words.

Clausal Argument Relations	Description
NSUBJ	Nominal subject
DOBJ	Direct object
IOBJ	Indirect object
CCOMP	Clausal complement
XCOMP	Open clausal complement
Nominal Modifier Relations	**Description**
NMOD	Nominal modifier
AMOD	Adjectival modifier
NUMMOD	Numeric modifier
APPOS	Appositional modifier
DET	Determiner
CASE	Prepositions, postpositions and other case markers
Other Notable Relations	**Description**
CONJ	Conjunct
CC	Coordinating conjunction

A dependency graph is a **directed graph *G = (V, A)*** where ***V*** is the set of vertices and ***A*** is the set of ordered pairs of vertices, called arcs. Arcs capture head-dependent and grammatical relationships between vertices. A dependency tree is a directed graph satisfying the following constraints:

1. there is a single **root** node that has no incoming arcs
2. each **vertex** except the root has one incoming arc
3. there exists a unique **path** from the root to each vertex in *V*

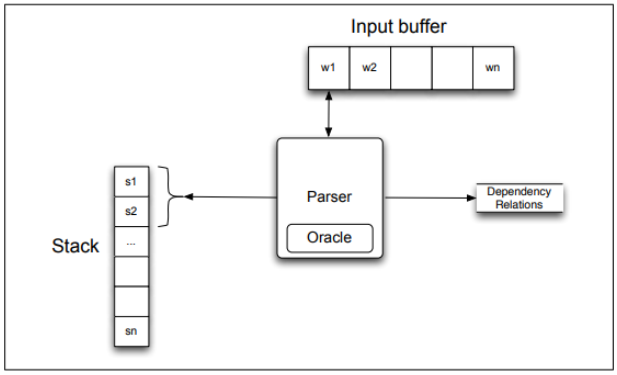
An arc from a head to a dependent is projective if there is a path from the head to each word between the head and dependent. A dependency tree is projective if all the arcs inside it are also projective.

<u>Transition-Based Dependency Parsing</u>

Shift-reduce dependency parsing uses a **context-free grammar, a stack, and a list of input tokens**. Input tokens are successively shifted onto the stack and the top two elements are matched against the grammar rules. When a match is found, a **dependency relation** is added to the parse tree.

- training data is provided as pairs of the form (configuration, transition)
- the configuration consists of a **stack**, **input tokens**, and **set of relations**

- transitions represent actions and labels
    - Shift: remove the word from the input and push it onto the stack
    - Left Arc: assert a head-dependent relation $A \rightarrow B$ where A is the head and B is the dependent
    - Right Arc: assert a head-dependent relation $A \leftarrow B$ where B is the head and A is the dependent



The algorithm for shift-reduce dependency parsing reduces the stack by creating dependency relations, until only the root node is left. The oracle analyses the top two items in the stack to determine if the parser should create a new relation or shift a new token onto the stack. The oracle has access to the following information:

- the current configuration with stack S and set of relations R_c
- the training data with vertices V and set of relations R_p

Given this information, the oracle selects a left arc if it produces a correct head-dependent relation, a right arc if it produces a correct head-dependent relation and all the dependents at the top of the stack have already been assigned, or a shift otherwise.

$$LeftArc(r)\text{: } \textbf{if } (S_1\ r\ S_2) \in R_p$$

$$RightArc(r)\text{: } \textbf{if } (S_2\ r\ S_1) \in R_p \textbf{ and } \forall r', w \mid (S_1\ r'w) \in R_p \textbf{ then } (S_1\ r'w) \in R_c$$

$$Shift\text{: } \textbf{otherwise}$$

```
function DEPENDENCYPARSE(words) returns dependency tree

  state ← {[root], [words], [] }  ; initial configuration
  while state not final
     t ← ORACLE(state)          ; choose a transition operator to apply
     state ← APPLY(t, state)    ; apply it, creating a new state
  return state
```

Features such as word forms, lemmas, and POS tags are used to train the oracle. Feature templates are abstract sets of predefined feature types to use during training. Features are denoted as *location.property* where *s* is the stack, *b* is the buffer, and *r* is the set of relations. *w* is a word form, *l* is a lemma, *t* is a POS tag, and *op* is the transition operator.

$$\langle s_1 \cdot w, op \rangle \; \langle s_2 \cdot w, op \rangle \; \langle s_1 \cdot t, op \rangle \; \langle s_2 \cdot t, op \rangle$$

$$\langle b_1 \cdot w, op \rangle \; \langle b_1 \cdot w, op \rangle \; \langle s_1 \cdot wt, op \rangle$$

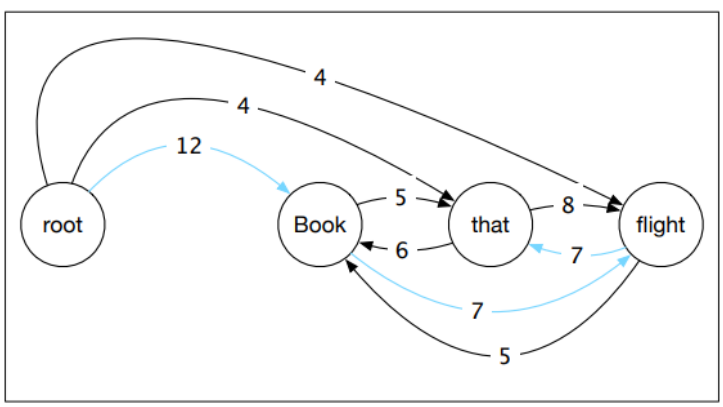
<u>Graph-Based Dependency Parsing</u>

Graph-based dependency parsing searches through a space of possible parse trees to maximise their score. The search space is represented as a directed graph which employs graph theory to search for optimal solutions. Given a sentence *S*, the parser searches for the best tree in the set of all possible trees G_S. Graph-based methods are useful as they can produce non-projective trees and have higher accuracy with long-distance dependencies.

$$\hat{T}(S) = argmax\ score(t, S)$$

$$score(t, S) = \sum_{e \in t} score(e)$$

Given an input sentence, the algorithm constructs a fully connected weighted directed graph where the **vertices** are the input words and the **edges** represent all possible head-dependent assignments. A **root** node is included with outgoing edges to all other vertices. The **weight** for each edge represents its score.



For each vertex in the graph, the algorithm selects the incoming edge with the highest score. If the resulting **subgraph *T = (V, F)*** is a spanning tree (has no cycles and each vertex except the root has one incoming edge) the tree is returned. Otherwise, the selection process continues. During cleanup, all weights are adjusted by subtracting the score of the maximum edge from the

score of all other edges entering that vertex. If any edges are equal to zero, the cycle is collapsed into a single node.

```
function MaxSpanningTree(G=(V,E), root, score) returns spanning tree

    F ← []
    T' ← []
    score' ← []
    for each v ∈ V do
        bestInEdge ← argmax_{e=(u,v)∈ E} score[e]
        F ← F ∪ bestInEdge
        for each e=(u,v) ∈ E do
            score'[e] ← score[e] − score[bestInEdge]

        if T=(V,F) is a spanning tree then return it
        else
            C ← a cycle in F
            G' ← Contract(G, C)
            T' ← MaxSpanningTree(G', root, score')
            T ← Expand(T', C)
            return T

function Contract(G, C) returns contracted graph

function Expand(T, C) returns expanded graph
```

The score for each edge can be reduced to a weighted sum of features. Common features include word forms, lemmas, POS tags, NER tags, embeddings, relations, dependencies, and dependency distances.

$$score(S,T) = \sum_{e \in t} score(S,e)$$

$$score(S,e) = \sum_{i=1}^{N} w_i f_i(S,e) = w \cdot f$$

Word Senses and WordNet

<u>Word Senses</u>

Words are ambiguous and can have different meanings. A word sense is a representation of one aspect of the meaning of a word.

The noun "bass" has 8 senses in WordNet.
1. bass^1 - (the lowest part of the musical range)
2. bass^2, bass part^1 - (the lowest part in polyphonic music)
3. bass^3, basso^1 - (an adult male singer with the lowest voice)
4. sea bass^1, bass^4 - (the lean flesh of a saltwater fish of the family Serranidae)
5. freshwater bass^1, bass^5 - (any of various North American freshwater fish with lean flesh (especially of the genus Micropterus))
6. bass^6, bass voice^1, basso^2 - (the lowest adult male singing voice)
7. bass^7 - (the member with the lowest range of a family of musical instruments)
8. bass^8 - (nontechnical name for any of numerous edible marine and freshwater spiny-finned fishes)

Relations between senses:

- Synonymity & Antonymity
  - Synonym: word which is identical to another
  - Antonym: word which is opposite to another
- Hyponymity & Hypernymity
  - Hyponym: word which is a subclass of another
  - Hypernym: word which is a superclass of another
- Meronymity & Holonymity
  - Meronym: word which is part of another
  - Holonym: word which is built from others

Relation	Also Called	Definition	Example
Hypernym	Superordinate	From concepts to superordinates	$breakfast^1 \rightarrow meal^1$
Hyponym	Subordinate	From concepts to subtypes	$meal^1 \rightarrow lunch^1$
Instance Hypernym	Instance	From instances to their concepts	$Austen^1 \rightarrow author^1$
Instance Hyponym	Has-Instance	From concepts to their instances	$composer^1 \rightarrow Bach^1$
Part Meronym	Has-Part	From wholes to parts	$table^2 \rightarrow leg^3$
Part Holonym	Part-Of	From parts to wholes	$course^7 \rightarrow meal^1$
Antonym		Semantic opposition between lemmas	$leader^1 \Longleftrightarrow follower^1$
Derivation		Lemmas w/same morphological root	$destruction^1 \Longleftrightarrow destroy^1$

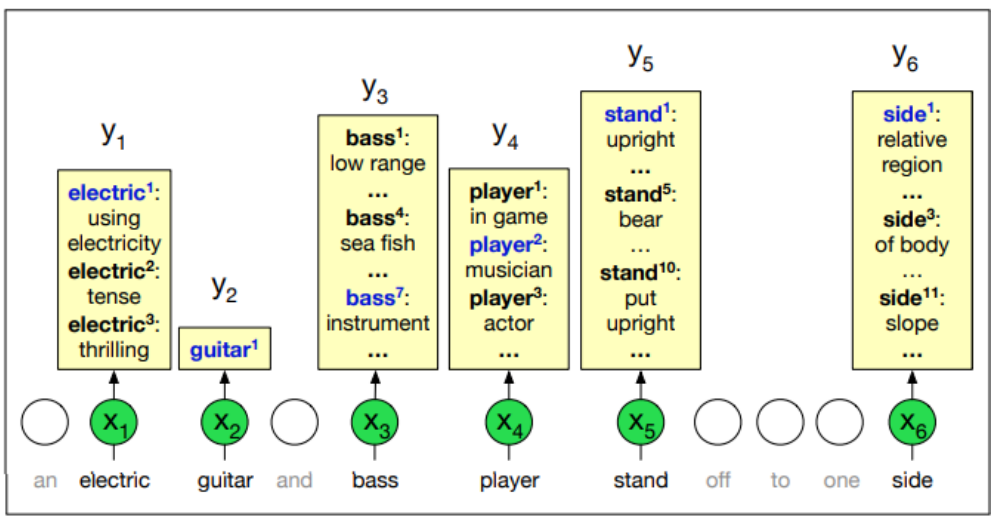
<u>WordNet</u>

WordNet is a **lexical database** containing semantic relations between words. Each word is described by a **gloss** (dictionary-style definition), list of synonyms for the sense, and usage examples. The set of synonyms for a sense is called a **synset**. The lexographic category for a synset is called a **supersense**.

Category	Example	Category	Example	Category	Example
ACT	*service*	GROUP	*place*	PLANT	*tree*
ANIMAL	*dog*	LOCATION	*area*	POSSESSION	*price*
ARTIFACT	*car*	MOTIVE	*reason*	PROCESS	*process*
ATTRIBUTE	*quality*	NATURAL EVENT	*experience*	QUANTITY	*amount*
BODY	*hair*	NATURAL OBJECT	*flower*	RELATION	*portion*
COGNITION	*way*	OTHER	*stuff*	SHAPE	*square*
COMMUNICATION	*review*	PERSON	*people*	STATE	*pain*
FEELING	*discomfort*	PHENOMENON	*result*	SUBSTANCE	*oil*
FOOD	*food*			TIME	*day*

Word Sense Disambiguation (WSD) is a **classification** task which predicts word senses in sentences. A WSD algorithm is typically trained using a **semantic concordance**, a corpus in which each word is labelled with its word sense. Given each noun, verb, adverb, or adjective in the test set, the algorithm will predict the correct sense from the possible senses provided in WordNet.

- **X** = sentence = sequence of labelled words
- **Y** = word senses = sequence of sense tags



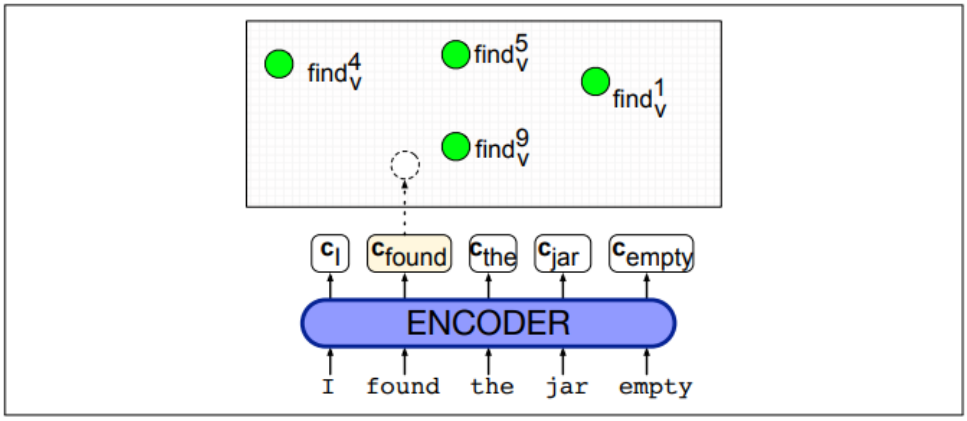
WSD algorithms are usually evaluated using the F1 score. A strong baseline involves selecting the **most frequent sense** for each word. Another baseline called **one-sense-per-discourse** involves using the same sense each time a word appears in text.

The most accurate form of WSD uses the nearest neighbour algorithm. During training, it passes each sentence into a labelled dataset to build a **contextual embedding** for each token.

For each token c_i of each sense c, it averages the embeddings to produce a **sense embedding** v_s for c:

$$v_s = \frac{1}{n} \sum_i c_i$$

The algorithm computes an **embedding** *t* for each word in the test set, then finds its nearest neighbour (cosine distance) from the training set.



For out-of-vocabulary (OOV) words, the algorithm can infer the missing embeddings using **supersenses**. It calculates the **sense embedding** for any higher-level word in the WordNet taxonomy by averaging the senses of its children. For each missing sense in WordNet $\hat{s} \in W$, let the **synset embeddings** be $S_{\hat{s}}$, the **hypernym synset embeddings** be $H_{\hat{s}}$, and the **supersense synset embeddings** be $L_{\hat{s}}$.

The embedding for a synset is the average of its sense embeddings:

$$if\ |S_{\hat{s}}| > 0, \boldsymbol{v}_{\hat{s}} = \frac{1}{|S_{\hat{s}}|} \sum \boldsymbol{v}_s, \forall\ \boldsymbol{v}_s \in S_{\hat{s}}$$

The embedding for a hypernym is the average of its synset embeddings:

$$if\ |H_{\hat{s}}| > 0, \boldsymbol{v}_{\hat{s}} = \frac{1}{|H_{\hat{s}}|} \sum \boldsymbol{v}_{syn}, \forall\ \boldsymbol{v}_{syn} \in H_{\hat{s}}$$

The embedding for a supersense is the average of its synset embeddings:

$$if\ |L_{\hat{s}}| > 0, \boldsymbol{v}_{\hat{s}} = \frac{1}{|L_{\hat{s}}|} \sum \boldsymbol{v}_{syn}, \forall\ \boldsymbol{v}_{syn} \in L_{\hat{s}}$$

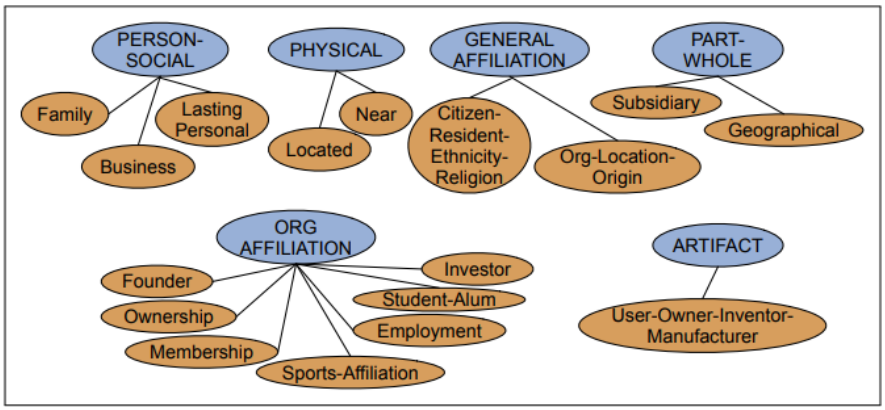
Information Extraction

Information extraction: process of converting unstructured text into structured data, such as a relational database or set of extracted tuples

Relation extraction: process of finding semantic relations among text entities, such as parent-child, part-whole or geospatial relations

Relation Extraction

A relation consists of a set of ordered tuples over named entities. ACE contains 17 relations and 7 entities:



Relations	Types	Examples
Physical-Located	PER-GPE	**He** was in **Tennessee**
Part-Whole-Subsidiary	ORG-ORG	**XYZ**, the parent company of **ABC**
Person-Social-Family	PER-PER	**Yoko**'s husband **John**
Org-AFF-Founder	PER-ORG	**Steve Jobs**, co-founder of **Apple**...

Wikipedia provides a database of relations contained in infoboxes. Relations appear as RDF (Resource Description Framework) triples appearing as subject-predicate-object expressions. DBpedia is an ontology derived from Wikipedia containing RDF triples.

Subject	Predicate	Object
Golden Gate Park	location	San Francisco

WordNet offers ontological relations between words and concepts such as hypernyms (is-a) and hyponyms (instance-of). The TACRED dataset contains relations triples associated with people and organisations, represented as labelled sentences.

Hearst patterns are hand-crafted lexico-syntactic patterns derived using regular expressions. These have high precision but low recall.

NP {, NP}* {,} (and|or) other NP_H	temples, treasuries, and other important civic buildings
NP_H such as {NP,}* {(or|and)} NP	red algae such as Gelidium
such NP_H as {NP,}* {(or|and)} NP	such authors as Herrick, Goldsmith, and Shakespeare
NP_H {,} including {NP,}* {(or|and)} NP	common-law countries, including Canada and England
NP_H {,} especially {NP}* {(or|and)} NP	European countries, especially France, England, and Spain

Supervised Learning

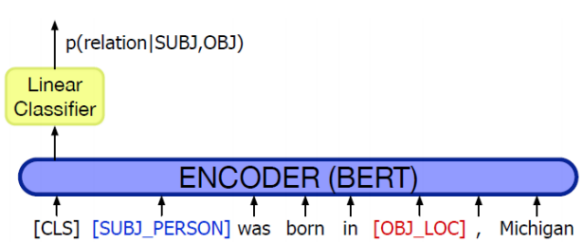
Supervised learning approaches are **trained** on an annotated corpus containing relations and entities, and the annotated texts are used to train **classifiers** to annotate unseen text. The supervised learning algorithm finds **pairs of named entities** in a sentence and **classifies** each pair.

- **X** = feature set for entity pair (embedding/one-hot vector)
- **Y** = prediction for the relation between *e1* and *e2*

```
function FINDRELATIONS(words) returns relations

    relations ← nil
    entities ← FINDENTITIES(words)
    forall entity pairs ⟨e1, e2⟩ in entities do
        if RELATED?(e1, e2)
            relations ← relations+CLASSIFYRELATION(e1, e2)
```

Feature-based classifiers typically use word features such as **embeddings**, **one-hot vectors**, **n-grams, lemmas**, named entity features such as **NER tags** and **entity levels**, and syntactical features such as **dependencies** or **dependency distances**.

Neural supervised relation classifiers perform supervised classification on words in a sentence. A typical **transformer-encoder** algorithm uses a pretrained encoder such as BERT to **encode** sentences with labels. The encoder replaces **subject** and **object** entities with NER tags to avoid overfitting. Neural classifiers perform well if the test set is similar to the training set. However, supervised models do not generalise well to different types of text.



Semi-Supervised Learning

Bootstrapping uses a small set of high quality **seed tuples** of the form *(r, e1, e2)* to find sentences containing both entities. It extracts and **generalises** the context around the entities to learn new patterns.

```
function BOOTSTRAP(Relation R) returns new relation tuples

    tuples ← Gather a set of seed tuples that have relation R
    iterate
        sentences ← find sentences that contain entities in tuples
        patterns ← generalize the context between and around entities in sentences
        newpairs ← use patterns to grep for more tuples
        newpairs ← newpairs with high confidence
        tuples ← tuples + newpairs
    return tuples
```

The bootstrapping algorithm also assigns **confidence values** to new tuples to avoid semantic drift. In semantic drift, an incorrect pattern leads to the introduction and propagation of incorrect tuples, causing the meaning of the relation to 'drift'.

<u>Distant Supervision</u>

Distant supervision combines the advantages of semi-supervised learning and bootstrapping. As distant supervision doesn't use a labelled training corpus, it isn't affected by semantic drift.

- **Training**
    - uses a large database containing many seed samples to create noisy pattern features
    - combines features in a supervised classifier
- **Relation Extraction**
    - finds relations from databases such as DBpedia and Freebase
    - a NER tagger is run on some text to extract sentences with matching relations
    - training instances are extracted in the form *(r, e1, e2)*
- **Prediction**
    - applies feature-based neural classification with a rich set of features

```
function DISTANT SUPERVISION(Database D, Text T) returns relation classifier C

  foreach relation R
    foreach tuple (e1,e2) of entities with relation R in D
      sentences ← Sentences in T that contain e1 and e2
      f ← Frequent features in sentences
      observations ← observations + new training tuple (e1, e2, f, R)
  C ← Train supervised classifier on observations
  return C
```

Unsupervised Learning

Unsupervised relation extraction finds relations without using labelled training data. This task is often called Open Information Extraction (Open IE). The ReVerb algorithm extracts relations from sentences in 4 steps:

1. Perform **POS and NER tagging** on a sentence *s*
2. For each verb in *s*, find the **longest phrase** *w* that starts with a verb and satisfies the constraints
3. For each phrase *w*, find the **nearest noun phrase** *x* to the left which is not a relative pronoun, *wh*-word, or existential word. Find the **nearest noun phrase** *y* to the right of the verb
4. Assign and return a **confidence value** *c* to the relation *r* = *(x,w,y)* using a **confidence classifier**

A relation is only accepted if it meets syntactic and lexical constraints. These ensure that it is a verb-initial sequence that may also include nouns. The constraints are based on a dictionary *D* that is used to prune irrelevant strings. This eliminates relations that do not occur often enough to form distinct examples.

V | VP | VW*P
V = verb particle? + adverb?
W = noun | pronoun | adjective | adverb | determiner
P = prep | particle | marker

Temporal Expression Extraction

Temporal expression extraction finds expressions containing dates and times. **Relative expressions** map to particular times through some reference point. **Durations** denote spans of time at varying levels of granularity.

Absolute	Relative	Durations
April 24, 1916	yesterday	four hours
The summer of '77	next semester	three weeks
10:15 AM	two weeks from yesterday	six days
The 3rd quarter of 2006	last quarter	the last three quarters

Rule-based systems can extract these expressions using lexical triggers such as nouns, verbs, and adjectives. **Sequence labelling** approaches use IOB tagging to label named entities in sentences. Features are extracted from tokens and their contexts, and a sequence labelling model is trained on the data. Typical features include word shape, POS tags, NER tags, context, and lexical triggers.

Feature	Explanation
Token	The target token to be labeled
Tokens in window	Bag of tokens in the window around a target
Shape	Character shape features
POS	Parts of speech of target and window words
Chunk tags	Base phrase chunk tag for target and words in a window
Lexical triggers	Presence in a list of temporal terms

Temporal normalisation is the process of mapping a temporal expression to a specific point in time or duration. Most approaches for this task are **rule-based** and match regular expressions such as *YYYY-MM-DD* and *HH:MM:SS*. The temporal anchor is the central point in time to which all other temporal expressions are compared.

Event extraction is used to identify events in texts. An **event** is an expression denoted an event or state that can be assigned to a temporal expression. Events are classified as **actions**, **states**, **reporting events** (say, tell, explain), **perception events** (see, observe), and more. Event extraction is usually performed using supervised learning. Sequence labelling models can use **IOB tagging** to classify event classes and attributes. **Feature-based models** use information such as POS tags, tense, and word forms.

Feature	Explanation
Character affixes	Character-level prefixes and suffixes of target word
Nominalization suffix	Character-level suffixes for nominalizations (e.g., *-tion*)
Part of speech	Part of speech of the target word
Light verb	Binary feature indicating that the target is governed by a light verb
Subject syntactic category	Syntactic category of the subject of the sentence
Morphological stem	Stemmed version of the target word
Verb root	Root form of the verb basis for a nominalization
WordNet hypernyms	Hypernym set for the target

<u>Template Filling</u>

A script is an **abstract** sequence of events, participants, and roles. Scripts can be represented as templates consisting of fixed sets of slots that take inputs called slot-fillers. The task of template filling involves finding documents that contain scripts and filling slots with slot-fillers extracted from the text.

- Template Recognition
  - training data consists predefined templates with annotated slot-fillers
  - matching scripts are found in some text
  - features are extracted from each labelled sentence that fills a slot
  - features include tokens, embeddings, word shapes, POS tags, and NER tags
  - a classifier is trained to find templates
- Role-Filler Extraction
  - training data contains roles such as *airline, amount*
  - features are extracted from each labelled sentence that contains a role

- a binary role classifier is trained on the labelled training data

Semantic Role Labelling

Semantic roles are relations between arguments and predicates, representing abstract concepts such as causation, experience, and effect. **Proto-agent** arguments display agent-like properties such as being violationally involved in an event and causing an event or change of state. **Proto-patient** arguments display patient-like properties such as undergoing a change of state and being affected by other participants.

Thematic Role	Definition
AGENT	The volitional causer of an event
EXPERIENCER	The experiencer of an event
FORCE	The non-volitional causer of the event
THEME	The participant most directly affected by an event
RESULT	The end product of an event
CONTENT	The proposition or content of a propositional event
INSTRUMENT	An instrument used in an event
BENEFICIARY	The beneficiary of an event
SOURCE	The origin of the object of a transfer event
GOAL	The destination of an object of a transfer event

<u>PropBank</u>

Propbank is a database of sentences annotated with **semantic roles**. Roles are defined separately for each possible **word sense**. Each verb sense has a specific set of rules of the form *Arg0, Arg1, Arg2…*

- *Arg0* - **proto-agent**
- *Arg1* - **proto-patient**
- *Argx* - *optional*

(19.11) **agree.01**

Arg0: Agreer
Arg1: Proposition
Arg2: Other entity agreeing

Ex1: [$_{Arg0}$ The group] *agreed* [$_{Arg1}$ it wouldn't make an offer].
Ex2: [$_{ArgM\text{-}TMP}$ Usually] [$_{Arg0}$ John] *agrees* [$_{Arg2}$ with Mary] [$_{Arg1}$ on everything].

PropBank provides additional arguments called *ArgMs* which represent **modifications** or **adjunct meanings**.

Modifier	Type	Meaning	Example

TMP	Temporal	**when** an action takes place	yesterday evening, now
LOC	Locative	**where** action takes place	at the museum in San Francisco
MNR	Manner	**how** an action is performed	clearly, with much enthusiasm
DIR	Directional	motion from source to goal	down to London
CAU	Cause	reason for an action	as a result of the interview
REC	Reciprocal	reflexes and reciprocals	himself, themselves, together
ADV	Adverbial	miscellaneous modifier	happily, she attended

<u>FrameNet</u>

FrameNet is a **semantic role-labelling** database that stores relations as frames. A frame is a background knowledge structure that connects a set of words using **semantic roles** called frame elements. For example, the role *change_position_on_a_scale* consists of words that indicate a change of position on a scale *attribute* from a starting point *initial_value* to an end point *final_value*.

(19.20) [$_{\text{ITEM}}$ Oil] *rose* [$_{\text{ATTRIBUTE}}$ in price] [$_{\text{DIFFERENCE}}$ by 2%].

(19.21) [$_{\text{ITEM}}$ It] has *increased* [$_{\text{FINAL_STATE}}$ to having them 1 day a month].

(19.22) [$_{\text{ITEM}}$ Microsoft shares] *fell* [$_{\text{FINAL_VALUE}}$ to 7 5/8].

(19.23) [$_{\text{ITEM}}$ Colon cancer incidence] *fell* [$_{\text{DIFFERENCE}}$ by 50%] [$_{\text{GROUP}}$ among men].

(19.24) a steady *increase* [$_{\text{INITIAL_VALUE}}$ from 9.5] [$_{\text{FINAL_VALUE}}$ to 14.3] [$_{\text{ITEM}}$ in dividends]

(19.25) a [$_{\text{DIFFERENCE}}$ 5%] [$_{\text{ITEM}}$ dividend] *increase*...

Core Roles	
ATTRIBUTE	The ATTRIBUTE is a scalar property that the ITEM possesses.
DIFFERENCE	The distance by which an ITEM changes its position on the scale.
FINAL_STATE	A description that presents the ITEM's state after the change in the ATTRIBUTE's value as an independent predication.
FINAL_VALUE	The position on the scale where the ITEM ends up.
INITIAL_STATE	A description that presents the ITEM's state before the change in the ATTRIBUTE's value as an independent predication.
INITIAL_VALUE	The initial position on the scale from which the ITEM moves away.
ITEM	The entity that has a position on the scale.
VALUE_RANGE	A portion of the scale, typically identified by its end points, along which the values of the ATTRIBUTE fluctuate.
Some Non-Core Roles	
DURATION	The length of time over which the change takes place.
SPEED	The rate of change of the VALUE.
GROUP	The GROUP in which an ITEM changes the value of an ATTRIBUTE in a specified way.

Semantic Role Labelling

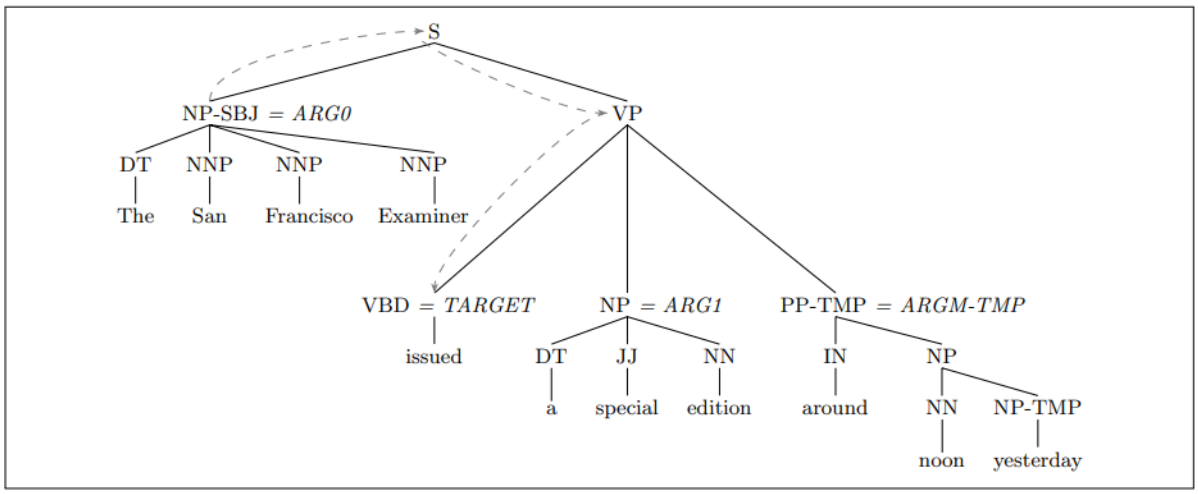
Semantic Role Labelling (SRL) is the task of extracting semantic roles for arguments and predicates in a sentence. A feature-based SRL algorithm extracts **features** from labelled words in a parse tree. A **classifier** is then trained to predict a **semantic role** for each constituent using the features.

- **X** = feature template for a parse tree
- **Y** = sequence of SRL tags

```
function SEMANTICROLELABEL(words) returns labeled tree

    parse ← PARSE(words)
    for each predicate in parse do
        for each node in parse do
            featurevector ← EXTRACTFEATURES(node, predicate, parse)
            CLASSIFYNODE(node, featurevector, parse)
```

The classification task can also be broken down into multiple steps:

- Pruning - removal of irrelevant words before training
- Identification - binary classification of each node as an argument to be labelled or *none*
- Classification - multiclass classification of constituents labelled as arguments
- Global Assignment - optimal role assignment using Viterbi or other constraint-based methods



Most SRL systems use a generalisation of core feature templates. For the sentence *"The San Francisco Examiner issued a special edition around noon yesterday"*, the following features could be used:

- the **governing predicate**, in this case the verb *issued*
- the **phrase type** of the constituent, in this case *NP* or *NP-SBJ*

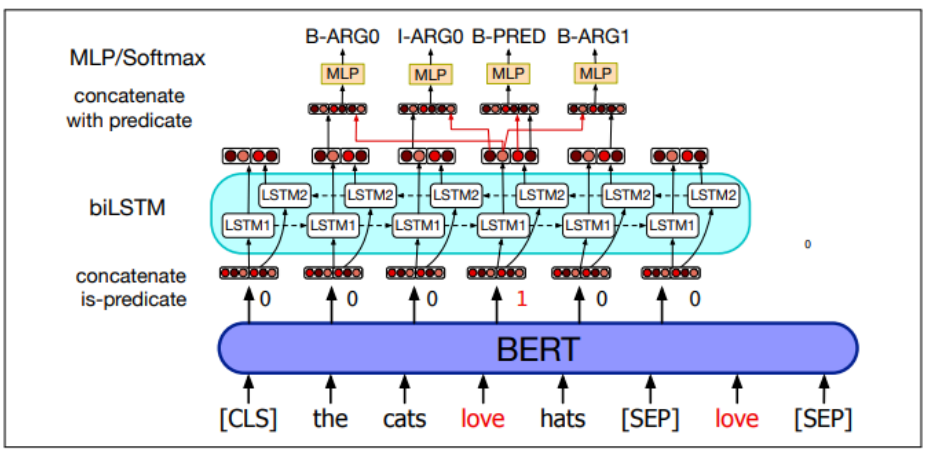
- the **headword** of the constituent, such as *Examiner*
- the **part of speech** of the headword, in this case *NNP*
- the **path** in the parse tree from the constituent to the predicate, such as *NP-S-VP-VBD*
- the **voice** of the clause, in this case *active*
- the **position** of the constituent, such as *before, after*
- the **subcategorisation** of the predicate, such as *VP → VBD NP PP*

Neural Networks for Semantic Role Labelling

A neural sequence labelling algorithm aims to compute the **most probable tag sequence** $\hat{y}$ given an input sequence *w*:

$$\hat{y} = argmax\ P(\mathbf{y}|\mathbf{w})$$

Each input word is mapped to pretrained BERT embeddings and associated with a flag indicating whether it is a predicate. The concatenated embeddings are passed through an LSTM network. The output embeddings and predicate embeddings are concatenated and passed through an MLP with a softmax function which outputs a probability for each tag.



Question Answering

Information Retrieval (IR) is the retrieval of media based on the information a user needs. The user provides a query to the retrieval system, which returns a set of relevant documents from a database. Documents include web pages, scientific papers, news articles, or paragraphs.

IR-Based Factoid QA

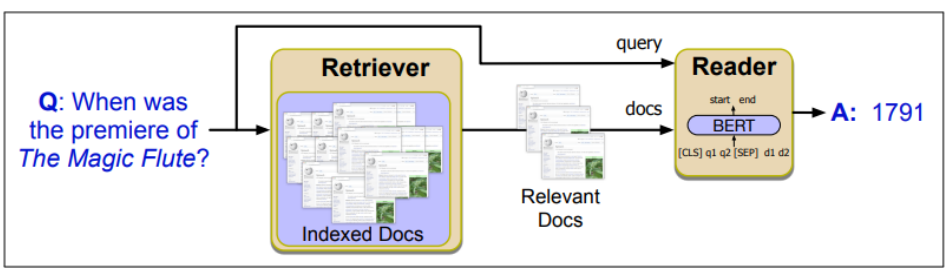
IR-Based QA/Open Domain QA answers questions by finding short text segments from documents such as web pages. Factoid questions always have short answers such as noun phrases and verb phrases.

Question	Answer
Where is the Louvre Museum located?	in Paris, France
What are the names of Odin's ravens?	Huginn and Muninn
What kind of nuts are used in marzipan?	almonds
What instrument did Max Roach play?	drums
What's the official language of Algeria?	Arabic

Datasets for IR-based QA are typically created by developing **reading comprehension datasets** containing tuples of the form *(passage, question, answer)*. This type of data can be used to train a reader to predict a span in the passage as the answer, given a passage and question.

- SQuAD contains 100,000 question-answer pairs based on 500+ articles from Wikipedia and associated questions whose answers are text spans from the passages
- HotspotQA contains 300,000 crowd-sourced questions and answers. Some of these are multi-hop questions requiring multiple documents to answer
- Natural Questions incorporates 300,000 real anonymised Google search queries, presented as a query, Wikipedia page, and annotated answer
- TyDi QA is a multilingual database over 11 languages containing 240,000 question and answer pairs from Wikipedia and crowd-sourcing

IR-Based QA used the retrieve and read model. In the **retrieval** stage, it finds relevant passages from a text database. In the **read** stage, a **neural reading comprehension** algorithm passes over each passage to find spans of text that are likely to answer the question.



Retrieve and read is typically performed by span labelling. The algorithm identifies a text span in a passage that constitutes an answer. A **neural algorithm** is given a question q containing n tokens $q_1 \ldots q_n$ and a passage p of m tokens $p_1 \ldots p_m$. It then computes the probability $P(a|q,p)$ that each possible text span a is the answer. If each span **starts** at position a_s and **ends** at position a_e, the probability can be estimated as $P(a|q,p) = P_{start}(a_s|q,p)\ P_{end}(a_e|q,p)$. For each token p_i in the passage, the algorithm computes the **start** and **end** probabilities p_{start} and p_{end}.

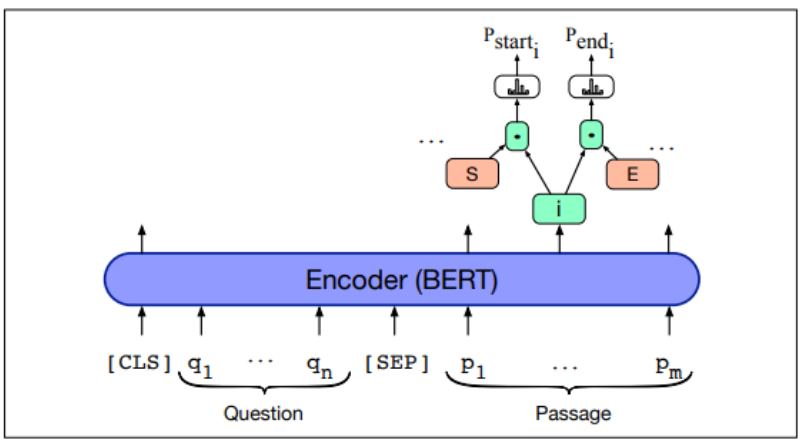
A standard **baseline** algorithm will pass the question and passage to an encoder such as BERT as strings separated with the [SEP] token. It then trains a network layer to predict the **start** and **end** position of the text span. These are represented by the vector embeddings *S* and *E*. For each output token p_i, the algorithm computes the **dot product** between *S* and p_i, then uses **softmax** to find its probability.

$$P_{start_i} = \frac{\exp(S \cdot p_i)}{\sum_j \exp(S \cdot p_j)}$$

$$P_{end_i} = \frac{\exp(E \cdot p_i)}{\sum_j \exp(E \cdot p_j)}$$

The score for a **span** from position *i* to *j* is $S \cdot p_i + E \cdot p_j$.The **output** with the highest probability is selected as the final model prediction. The model is trained using log-loss with respect to the **start** and **end** positions for each text span.

$$L = -\log P_{start_i} - \log P_{end_i}$$



Entity Linking

Entity linking is the task of associating a mention in text with a representation of a real-world entity. TAGME is a simple baseline algorithm which uses data from Wikipedia pages. It first creates a catalog of Wikipedia pages and indexes them in a standard IR engine. For each Wikipedia page *e*, the algorithm computes an **in-link** count, which is the total number of links from other pages that point to *e*. Finally, the algorithm requires an anchor dictionary. The anchor dictionary contains the anchor texts for each page. These are hyperlinked text spans from other pages that point to the current page *e*.

Given an input question, TAGME detects mentions by querying the anchor dictionary for each token sequence. If the mention is ambiguous (points to more than one page), the algorithm

disambiguates the mention by calculating a score based on two metrics. The prior probability *P(e|a)* is the probability that the span refers to the entity. For each page, the probability that an anchor *a* points to *e* is the ratio of pages linked to *e* with anchor text *a* and the total number of occurrences of *a* as an anchor:

$$prior\ probability(a \rightarrow e) = P(e|a) = \frac{count(a \rightarrow e)}{link(a)}$$

Given a question *q*, the relatedness for an entity *e* and anchor *a* is the weighted average relatedness between *e* and all other entities in *q*. *in(x)* is the set of Wikipedia pages pointing to *x* and *W* is the set of all Wikipedia pages in the collection. To calculate the relatedness between two entities *A* and *B*:

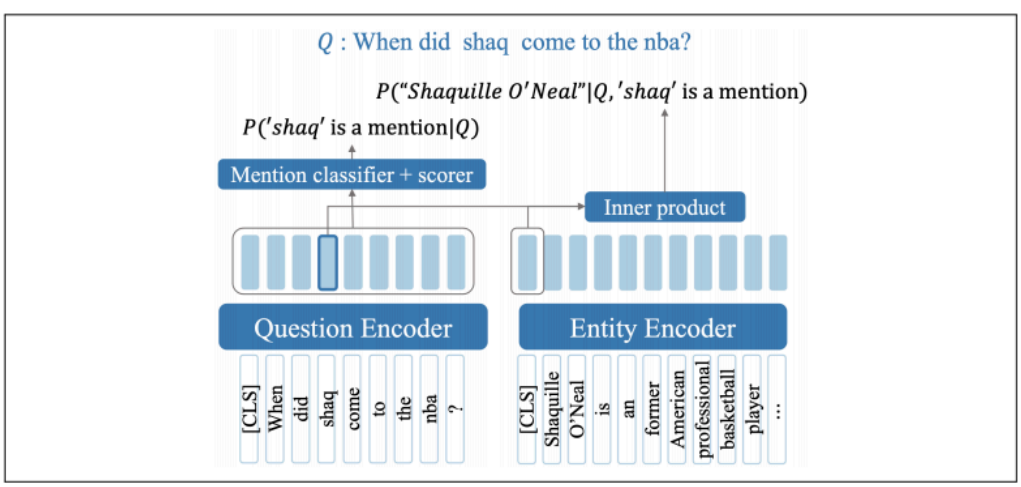
$$relatedness(A, B) = \frac{\log(\max(|\text{in}(A)|, |\text{in}(B)|)) - \log(|\text{in}(A)| \cap |\text{in}(B)|)}{\log(|W|) - \log(\min(|\text{in}(A)|, |\text{in}(B)|))}$$

To compute a score for each entity, TAGME combines **relatedness** and **prior probability** to select entities with high **relatedness** scores, then selecting the entity with the highest **prior probability** from this set.

<u>Neural Entity Linking</u>

A neural graph-based linking algorithm encodes a **candidate span**, encodes an **entity**, and computes the **dot product** between the encodings. This allows **embeddings** for all the entities in the knowledge base to be pre-computed and cached. Given a question *q* and a set of candidate mentions from Wikipedia text, the algorithm outputs tuples of the form (e, m_s, m_e) where *e* = entity, m_s = mention start, m_e = mention end.

- **X** = question, entity candidate
- **Y** = entity mention classifier + entity linker



To get a *h*-dimensional embedding for each question token, the algorithm runs the question through BERT.

$$[\boldsymbol{q}_1 \dots \boldsymbol{q}_n] = \mathrm{BERT}([\mathrm{CLS}]q_1 \dots q_n[\mathrm{SEP}])$$

It then computes the probability of each span *[i,j]* in *q* being an **entity mention**. First it must compute the score for the span being the start or end of a mention.

$$s_{start}(i) = \boldsymbol{w}_{start} \cdot \boldsymbol{q}_i,$$

$$s_{end}(j) = \boldsymbol{w}_{end} \cdot \boldsymbol{q}_j$$

w_{start} and w_{end} are vectors learned during training. Another trained embedding $w_{mention}$ is used to compute a score for each token being part of a mention.

$$s_{mention}(t) = \boldsymbol{w}_{mention} \cdot \boldsymbol{q}_t$$

The three scores are combined to create the mention probability.

$$p([i,j]) = \sigma\left(s_{start}(i) + s_{end}(j) + \sum_{t=i}^{j} s_{mention}(t)\right)$$

The algorithm links mentions to entities by computing dot product similarity between each entity *e* and span *[i,j]*.

$$\boldsymbol{y}_{i,j} = \frac{1}{(j-i+1)} \sum_{t-i}^{j} \boldsymbol{q}_t$$

$$s(e,[i,j]) = \boldsymbol{x}_e \boldsymbol{y}_{i,j}$$

Finally, it takes a softmax distribution over entities for each span.

$$p(e,[i,j]) = \frac{\exp(s(e,[i,j]))}{\sum \exp(s(e',[i,j]))}$$

<u>Evaluation of Factoid Answers</u>

The mean reciprocal rank (MRR) compares a set of **gold answers** and a **ranked output of answers** by calculating the reciprocal. The score of a system is the **average** of the MRR scores across questions in a set.

$$\mathrm{MRR} = \frac{1}{N}_{i=1} \sum_{rank_i \neq 0}^{N} \frac{1}{rank_i}$$

Reading comprehension systems are often evaluated using two metrics:

- Exact Match - the percentage of predicted answers that match the gold answer exactly
- F1 Score - the average overlap between predicted and gold answers

---